

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Теоретические основы баз данных
Отчет о выполнении курсовой работы

**Реализация базы данных для предметной
области «Геомониторинг автопарка
каршеринга»**

Студент,
группы 5130201/40003

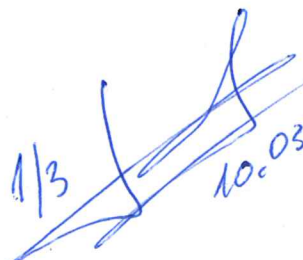
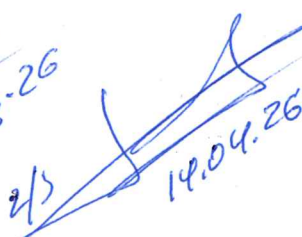
Адиатуллин Т. Р.



Руководитель,
доцент к.т.н., доцент

Попов С. Г.



1/3  10.03.26
2/3  14.04.26

«19» 05 2026 г.

Санкт-Петербург, 2026

Реферат

Предметной областью данной работы является «Геомониторинг автопарка каршеринга». Основная проблема состоит в том, что автомобили каршеринга подвержены угонам и несанкционированному использованию: без единой системы хранения треков перемещения, тревожных событий и парковочных сессий невозможно установить местонахождение автомобиля, отследить его маршрут и среагировать на тревожные события.

В рамках работы спроектирована и реализована реляционная база данных, хранящая сведения об автомобилях, сотрудниках, зонах парковки, тревожных событиях и треках перемещения. Сформированы и выполнены 9 запросов.

Для заполнения базы данных использовался `seeder`, написанный на языке Go с применением библиотек **pgx/v5** для взаимодействия с PostgreSQL и **golang-migrate** для применения миграций; СУБД PostgreSQL 14 развёртывалась в контейнере Docker; диаграммы результатов запросов построены средствами Python (**matplotlib**).

Разработанная база данных может служить основой для построения систем управления городскими парковками, платформ мониторинга тревожных событий и также сервисов каршеринга или приложений доставки.

Содержание

Введение	4
1 Постановка задачи	5
2 Аналитика	6
2.1 Описание предметной области	6
2.2 Цели функционирования базы данных	9
2.3 Сущности предметной области и их атрибуты	9
2.4 Перечень ролей	12
2.5 Схема связи объектов предметной области	14
3 Проектирование	15
3.1 Таблицы с описанием таблиц базы данных	15
3.2 Лингвистическое описание схемы базы данных	21
3.3 Исходный текст программы генерации схемы базы данных	26
3.4 Исходный текст программы заполнения базы данных	26
4 Программирование	28
4.1 Запрос 1.1	28
4.2 Запрос 1.2	30
4.3 Запрос 2	32
4.4 Запрос 3	34
4.5 Запрос 4	37
4.6 Запрос 5	40
4.7 Запрос 6	43
4.8 Запрос 7	45
4.9 Запрос 8	47
4.10 Запрос 9	50
Заключение	52
Список использованных источников	53
Приложение А	54
Приложение Б	60

Введение

База данных — совокупность данных, хранимых как в соответствии со схемой данных так и в произвольном порядке, манипулирование которыми выполняют в соответствии с правилами средств моделирования данных. Система управления базами данных — совокупность программных и лингвистических средств общего или специального назначения, обеспечивающих управление созданием и использованием баз данных.

Чтобы создать базу данных, сначала проектируют её структуру: определяют, какие данные хранить и как они связаны между собой. Затем схему переносят в СУБД и заполняют данными. Если база спроектирована правильно, данные не дублируются, запросы выполняются быстро и всё остаётся в порядке даже когда с базой работают одновременно несколько человек.

Данная работа посвящена проектированию и реализации реляционной базы данных для предметной области «Геомониторинг автопарка каршеринга». Современные сервисы каршеринга управляют сотнями автомобилей, и отслеживание их местонахождения, контроль парковочных сессий, а также своевременная реакция на тревожные события — угоны, выезды за разрешённые зоны, потерю связи — требуют надёжной системы хранения и быстрого доступа к данным.

В качестве СУБД используется PostgreSQL 14; заполнение базы данных реализовано на языке Go с использованием библиотек **pgx/v5** и **golang-migrate**.

1. Постановка задачи

Цель работы: реализация функционирующей реляционной базы данных для предметной области «Геомониторинг автопарка каршеринга» и выполнение заданных запросов.

Задачи работы:

- проанализировать предметную область;
- определить цели функционирования базы данных;
- выявить сущности, их атрибуты и связи;
- построить ER-диаграмму в нотации Чена;
- спроектировать схему базы данных и описать таблицы;
- перенести схему в СУБД PostgreSQL;
- реализовать программу заполнения базы данных тестовыми данными;
- выполнить запросы к базе данных;
- построить графическое представление результатов запросов.

2. Аналитика

2.1. Описание предметной области

Геомониторинг автопарка каршеринга - это постоянная система контроля местонахождения автомобилей, которые сдаются в краткосрочную аренду. Каршеринг - это форма использования транспортных средств, при которой клиент получает временный доступ к автомобилю на ограниченный промежуток времени для передвижения по городу. Автопарк каршеринга включает все автомобили, находящиеся в собственности компании и доступные для аренды клиентами. Поскольку такие автомобили эксплуатируются каждый день, перемещаются по большой территории и используются разными водителями, возникает необходимость в непрерывном наблюдении за их местонахождением и техническим состоянием.

Основой геомониторинга является определение местоположения автомобиля в реальном времени. Для этого в каждый автомобиль устанавливается телематический терминал - устройство, предназначенное для сбора и передачи данных. Терминал оснащён спутниковым модулем, который работает одновременно с навигационными системами GPS и ГЛОНАСС, что повышает точность и надёжность определения координат.

GPS - это глобальная спутниковая навигационная система. Она работает так: над Землёй летает группа из 32 спутников на высоте примерно 20 000 километров. Каждый спутник постоянно передаёт радиосигнал, в котором содержится информация о времени отправки сигнала и о том, где именно находится спутник. Приёмник в машине ловит эти сигналы и измеряет, сколько времени сигнал шёл от спутника до приёмника. Зная скорость радиоволн (скорость света), приёмник вычисляет расстояние до каждого спутника. Чтобы точно определить своё положение на Земле, приёмнику нужно поймать сигнал минимум от трех спутников одновременно.

Координаты определяются в системе WGS-84 (World Geodetic System 1984) - это мировой стандарт координат для GPS. В этой системе положение любой точки на Земле описывается тремя числами: широта (насколько далеко от экватора), долгота (насколько далеко от нулевого меридиана) и высота. Система WGS-84 представляет Землю как эллипсоид - немного сплюснутый шар. Центр системы координат WGS-84 совпадает с центром масс Земли, а математические параметры эллипсоида подобраны так чтобы максимально точно описать форму планеты. В городе GPS определяет координаты с точностью 5-15 метров, этого достаточно для каршеринга. В России раньше использовали другие системы координат - СК-42 (система координат 1942 года, Пулково 42) и СК-95 (ПЗ-90, используется в ГЛОНАСС). Но для GPS используется именно WGS-84, потому что это единый стандарт для всего мира, и не нужно пересчитывать координаты при переезде из одной страны в другую.

Чтобы система работала лучше, в телематический терминал встроена ещё одна технология - инерциальная система навигации (ИНС). Это набор датчиков:

акселерометры (измеряют ускорение машины) и гироскопы (измеряют повороты). Когда машина заезжает в тоннель или под мост, сигнал со спутников может пропасть. В этот момент инерциальная система продолжает следить за движением машины - она знает последнюю известную точку, скорость и направление, и может примерно посчитать, где машина находится сейчас. Это называется аппроксимация координат - система делает приблизительный расчёт положения на основе данных о движении. Когда сигнал со спутников снова появляется, система GPS корректирует эту погрешность и снова начинает показывать точные координаты. Такая комбинация спутниковой навигации и инерциальных датчиков позволяет непрерывно отслеживать машину даже в сложных городских условиях.

Собранные данные о местоположении автомобиля передаются на сервер компании по сотовым сетям связи 4G (LTE). Для этого в терминале стоит SIM-карта с IoT-тарифом. IoT (Internet of Things) - это когда разные устройства подключены к интернету и автоматически обмениваются данными без участия человека. Например, холодильник может сам заказывать продукты, когда они заканчиваются. В каршеринге телематический терминал - это IoT-устройство: он сам собирает информацию где машина, с какой скоростью едет, сколько топлива осталось и автоматически отправляет эти данные на сервер компании. IoT-тарифы сотовых операторов специально сделаны для таких устройств: они работают круглосуточно, стоят дешево, работают в любом месте города без разрывов связи и очень надёжные. Данные отправляются либо раз в какое то количество времени, либо когда происходит что-то важное, например машина превысила скорость, резко затормозила, произошёл удар, началась или закончилась аренда.

В случае потери связи по мобильной сети телематический терминал сохраняет данные в свою внутреннюю память обычно от 512 МБ до 2 ГБ. Этого хватает, чтобы сохранить информацию о поездках за несколько дней. Система на сервере компании не паникует сразу, если связь с машиной пропала - она ждёт 5-10 минут, потому что связь может временно пропасть из-за помех или плохого покрытия сети. Если связь не появляется после этого времени, внешняя серверная система автоматически создаёт уведомление и отправляет его операторам. Важный момент: анализ ситуации и решение о том, что нужно действовать, принимает не сама машина, а серверная система компании. Сервер постоянно следит за всеми машинами автопарка, проверяет поступающие данные, сравнивает их с нормальными показателями и создаёт тревожные сообщения для операторов, если что-то идёт не так.

Операторы колл-центра связываются с клиентом, который использует машину, чтобы выяснить, что случилось. Если клиент не отвечает или обнаружена неисправность, на последнее известное место, где была машина, отправляется сервисная бригада. Эти специалисты ищут машину, заправляют её, заряжают или устраняют мелкие поломки.

Системой геомониторинга пользуются различные категории людей. Клиенты каршеринга через мобильное приложение видят автомобили на карте, их

координаты и уровень топлива или заряда, что позволяет выбрать подходящую машину. Операторы колл-центра используют систему для контроля текущих поездок, помощи клиентам и обработки нештатных ситуаций. Передвижные сервисные группы получают задания с указанием конкретных автомобилей и их координат для технического обслуживания. Аналитики компании используют данные для анализа загрузки автопарка, выявления зон дефицита или избытка автомобилей и оптимизации распределения машин по городу.

Ключевой процесс 1 - построение трека движения автомобиля. Телематический терминал непрерывно фиксирует координаты автомобиля и отправляет их на сервер с интервалом 10-15 секунд. Серверная система принимает каждую новую точку с координатами, временной меткой и последовательно записывает все полученные точки в базу данных, формируя трек движения. Для каждого автомобиля ведётся отдельный трек, который показывает весь маршрут за определённый период времени. По этим данным система строит линию на карте, соединяющую все точки в хронологическом порядке, что позволяет визуально увидеть, где именно ездила машина. В этом процессе участвуют несколько ролей: клиент каршеринга непосредственно управляет автомобилем во время поездки, а телематический терминал в фоновом режиме собирает координаты и отправляет их на сервер компании, оператор колл-центра в реальном времени наблюдает за движением автомобиля на карте и может отследить текущее местоположение любой машины из автопарка если поступил запрос от клиента или возникла нештатная ситуация, аналитик компании работает с архивными треками за прошедшие периоды и строит отчёты по пробегам, популярным маршрутам и зонам активности, а администратор системы следит за корректностью поступления данных от всех терминалов и настраивает параметры записи треков в базу данных. Трек сохраняется в архиве и используется для анализа: расчёта пробега, определения времени нахождения в конкретных зонах, проверки соблюдения разрешённой территории эксплуатации, расследования инцидентов и споров с клиентами. Результатом процесса является полная история перемещений каждого автомобиля с точной географической привязкой, доступная для просмотра и анализа операторами и аналитиками компании.

Ключевой процесс 2 - определение зон парковки и контроль завершения аренды. Когда клиент завершает аренду через мобильное приложение, серверная система фиксирует финальные координаты автомобиля и начинает анализ местоположения. Программа на сервере проверяет, находится ли автомобиль в пределах разрешённой зоны завершения аренды - это может быть вся территория города или определённые районы в зависимости от тарифа. Далее система определяет тип места парковки: на улице в зоне бесплатной парковки, на платной парковке, во дворе жилого дома, на территории торгового центра. Для этого сервер сравнивает координаты с заранее загруженными границами парковочных зон. Если автомобиль припаркован в запрещённом месте вне парковочной зоны, сервер отклоняет завершение аренды и уведомляет клиента о необходимости переместить автомобиль в правильное место. В этом процессе ключевую роль играет клиент каршеринга, который паркует автомобиль в

конце поездки и через мобильное приложение нажимает кнопку завершения аренды, после чего приложение показывает ему результат проверки координат и либо подтверждает завершение либо просит переставить машину в другое место, оператор колл-центра вмешивается в процесс только если у клиента возникли вопросы по парковке или система зафиксировала нарушение правил, администратор системы заранее загружает в базу данных актуальные границы разрешённых парковочных зон и обновляет их при изменении городских правил парковки, а аналитик компании изучает статистику по местам завершения аренды и выявляет зоны где скапливается слишком много автомобилей или наоборот где машин не хватает чтобы предложить корректировки тарифов или бонусов за парковку в нужных местах. Также система проверяет расстояние от припаркованного автомобиля до других машин автопарка - если в одном месте скапливается слишком много автомобилей, система может предложить клиенту скидку за парковку в дефицитной зоне или наоборот, доплату за парковку в зоне избытка. После успешной проверки система фиксирует координаты парковки, делает запись в базу данных и отображает автомобиль на карте как доступный для следующего клиента. Результатом процесса является корректное размещение автомобиля на парковке в разрешённой зоне с зафиксированными координатами, что обеспечивает равномерное распределение автопарка по городу и соблюдение правил парковки.

2.2. Цели функционирования базы данных

1. Хранение треков движения автомобилей. База данных должна сохранять все координаты автомобилей с временными метками, чтобы система могла строить треки движения.
2. Хранение зон парковки. База данных должна хранить границы зон парковки, чтобы система могла проверять правильность завершения аренды и предлагать клиентам корректные места для парковки.
3. Хранение тревожных событий связанных с автомобилем. База данных должна фиксировать все инциденты, такие как превышение скорости, резкое торможение, удар, попытки угона, чтобы операторы могли оперативно реагировать на нештатные ситуации.
4. Хранение обращений персонала к данным геомониторинга. База данных должна сохранять все запросы операторов, администраторов к данным геомониторинга, чтобы можно было отслеживать кто и когда обращался к информации о местоположении автомобилей и какие данные запрашивал.

2.3. Сущности предметной области и их атрибуты

1. **Автомобиль** – транспортное средство, которым владеет каршеринг.

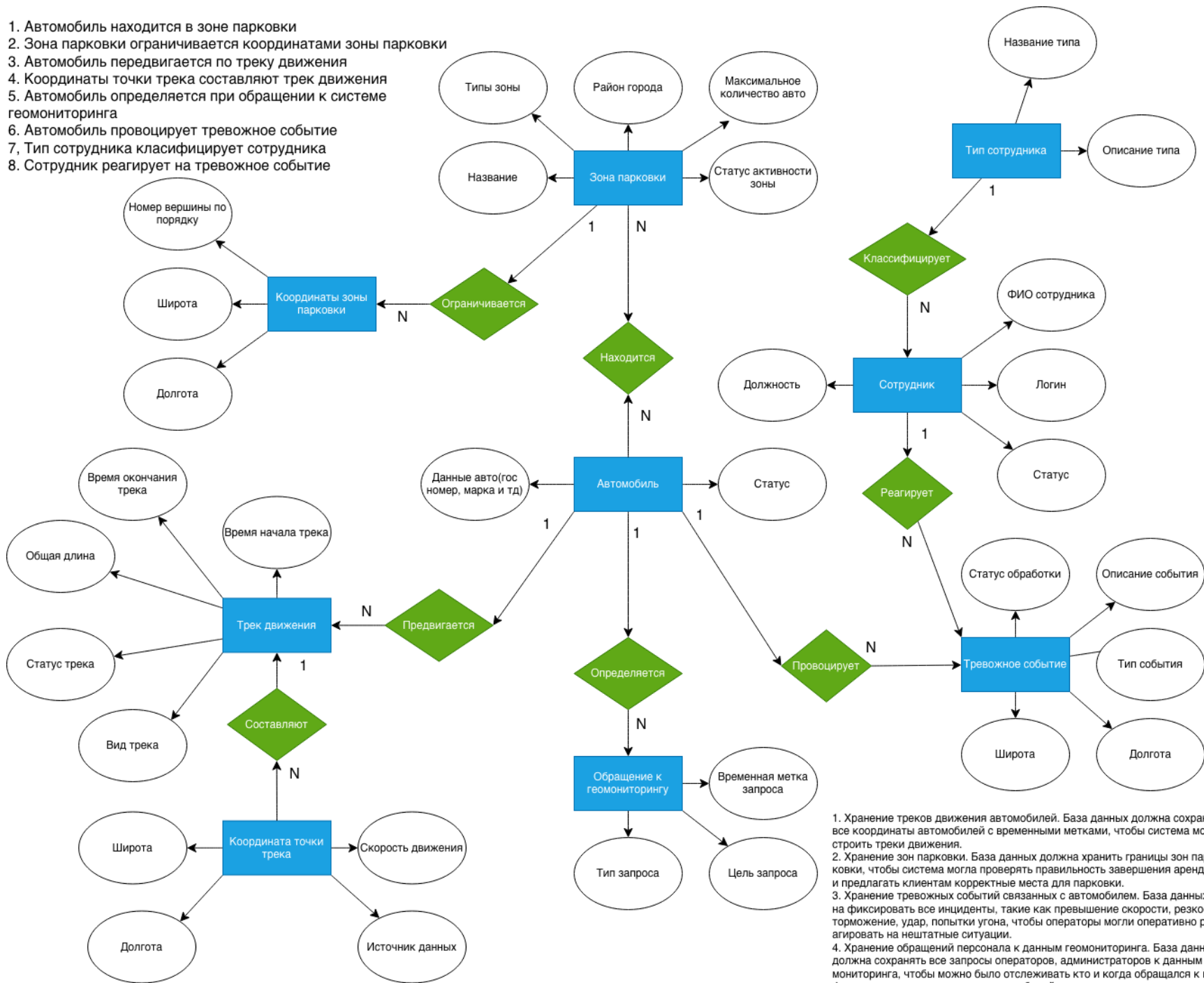
- Гос номер – государственный регистрационный знак автомобиля
 - Марка – производитель автомобиля
 - Модель – название модели автомобиля
 - Год выпуска – год производства автомобиля
 - VIN-номер – уникальный идентификационный номер транспортного средства
 - Текущий статус – состояние автомобиля (свободен, в аренде, на обслуживании, неисправен)
2. **Трек движения** – траектория перемещения автомобиля внутри одной поездки.
- Время начала трека – момент начала записи маршрута движения
 - Время окончания трека – момент завершения записи маршрута движения
 - Статус трека – состояние записи (активный, завершён, архивный)
 - Вид трека – категория маршрута (пользовательский, сервисный)
3. **Координаты точки трека** – географическая точка с временной меткой на маршруте автомобиля.
- Широта – географическая широта в системе координат WGS-84
 - Долгота – географическая долгота в системе координат WGS-84
 - Скорость движения – скорость автомобиля в момент измерения в км/ч
 - Источник данных – система определения координат (GPS, инерциальная система)
4. **Зона парковки** – территория для завершения аренды и парковки автомобилей.
- Название – наименование зоны парковки
 - Тип зоны – категория парковки (бесплатная, платная, запрещённая)
 - Район города – административный район расположения зоны
 - Максимальное количество автомобилей – лимит машин которые могут одновременно находиться в зоне
 - Статус активности зоны – признак доступности зоны для использования
5. **Координаты зоны парковки** – вершины полигона определяющего границы зоны парковки на карте.
- Номер вершины по порядку – порядковый номер точки в контуре полигона

- Широта – географическая широта вершины в системе координат WGS-84
 - Долгота – географическая долгота вершины в системе координат WGS-84
6. **Тревожное событие** – инцидент или нештатная ситуация зафиксированная системой мониторинга автомобиля.
- Тип события – категория инцидента (потеря связи, превышение скорости, резкое торможение, удар, попытка угона, выезд за границу)
 - Широта – географическая широта места события
 - Долгота – географическая долгота места события
 - Описание события – текстовая информация о произошедшем инциденте
 - Статус обработки – состояние реагирования на событие (новое, в работе, закрыто)
7. **Сотрудник** – работник компании имеющий доступ к системе геомониторинга автопарка.
- ФИО – полное имя сотрудника (фамилия, имя, отчество)
 - Логин – учётная запись для входа в систему
 - Статус – текущее состояние сотрудника (активен, в отпуске, уволен, заблокирован)
 - Должность – занимаемая позиция в компании
8. **Тип сотрудника** – категория работника определяющая его роль и права доступа.
- Название типа – наименование категории (оператор колл-центра, администратор системы, сервисный специалист, аналитик)
 - Описание – текстовое пояснение роли и функций данного типа сотрудника
9. **Обращения к геомониторингу** – обращение сотрудника к системе для получения данных о местоположении.
- Тип запроса – категория запрашиваемых данных (текущие координаты, история событий)
 - Цель запроса – причина обращения к данным о местоположении автомобиля

2.4. Перечень ролей

1. **Клиент каршеринга** - человек, который арендует автомобиль. В приложении он может найти машину на карте, начать поездку и потом её закончить.
2. **Оператор колл-центра** - сотрудник, который следит за всеми машинами на карте, помогает клиентам с вопросами по аренде и реагирует на нештатные ситуации.
3. **Сервисная бригада** - специалисты, которые выполняют техническое обслуживание, перегонку автомобилей. Они получают задания от системы, которая указывает конкретные машины и их координаты для обслуживания, заправки или ремонта.
4. **Администратор системы** - человек, который отвечает за настройку и поддержку системы геомониторинга. Он загружает данные о зонах парковки, следит за корректностью поступления данных от терминалов и управляет правами доступа сотрудников.

1. Автомобиль находится в зоне парковки
2. Зона парковки ограничивается координатами зоны парковки
3. Автомобиль передвигается по треку движения
4. Координаты точки трека составляют трек движения
5. Автомобиль определяется при обращении к системе геомониторинга
6. Автомобиль провоцирует тревожное событие
7. Тип сотрудника классифицирует сотрудника
8. Сотрудник реагирует на тревожное событие



1. Хранение треков движения автомобилей. База данных должна сохранять все координаты автомобилей с временными метками, чтобы система могла строить треки движения.
2. Хранение зон парковки. База данных должна хранить границы зон парковки, чтобы система могла проверять правильность завершения аренды и предлагать клиентам корректные места для парковки.
3. Хранение тревожных событий связанных с автомобилем. База данных должна фиксировать все инциденты, такие как превышение скорости, резкое торможение, удар, попытки угона, чтобы операторы могли оперативно реагировать на нештатные ситуации.
4. Хранение обращений персонала к данным геомониторинга. База данных должна сохранять все запросы операторов, администраторов к данным геомониторинга, чтобы можно было отслеживать кто и когда обращался к информации о местоположении автомобилей и какие данные запрашивал.

Рис. 1: ER-диаграмма

2.5. Схема связи объектов предметной области



Рис. 2: Схема связи объектов предметной области

3. Проектирование

3.1. Таблицы с описанием таблиц базы данных

В таблицах 1–19 представлены атрибуты таблиц базы данных геомониторинга. Каждая таблица с описанием содержит колонки: номер по порядку, имя атрибута на русском и английском языках, тип атрибута, при необходимости со значением размера, значение по умолчанию, признак ключа при наличии, для внешнего ключа ссылка на таблицу, примечание.

Таблица 1: Типы сотрудников

Тип сотрудника – employee_type							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	тип_сотрудника_id	employee_type_id	SERIAL	-	PK	-	NOT NULL
2	название_типа	name	VARCHAR(30)	-	UQ	-	NOT NULL
3	описание	description	TEXT	NULL	-	-	NULL

Обоснование: название типа сотрудника — VARCHAR(30). Перечень ролей закрытый и фиксированный: «Оператор колл-центра», «Администратор системы», «Сервисный специалист», «Аналитик» — самое длинное значение занимает 21 символа. Выбрано 30 как ближайшая граница выше наблюдаемого максимума.

Таблица 2: Таблица статусов сотрудника

Статус сотрудника – employee_status							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	статус_id	employee_status_id	SERIAL	-	PK	-	NOT NULL
2	название	name	VARCHAR(20)	-	UQ	-	NOT NULL

Обоснование: название статуса сотрудника — VARCHAR(20). Перечень статусов закрытый: «Активен», «В отпуске», «Уволен», «Заблокирован» — максимум 12 символов. Любой возможный новый статус, например «На испытании» (13 символов), также укладывается в выбранный предел.

Таблица 3: Таблица сотрудников

Сотрудник – employee							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	сотрудник_id	employee_id	SERIAL	-	PK	-	NOT NULL
2	тип_сотрудника_id	employee_type_id	INT	-	FK	empl_type	NOT NULL
3	фio	full_name	VARCHAR(100)	-	-	-	NOT NULL
4	логин	login	VARCHAR(30)	-	UQ	-	NOT NULL
5	статус_id	employee_status_id	INT	-	FK	employee_status	NOT NULL

Обоснование: ФИО — VARCHAR(100). Согласно Приказу ФМС России от 30.06.2011 № 279 «Об утверждении Правил и способа формирования машиночитаемой записи в паспорте гражданина Российской Федерации», верхняя строка машиночитаемой записи отводит под ФИО 39 символов — это аппаратное ограничение физического бланка паспорта. Поскольку в системе ФИО хранится одной строкой в кириллице, а не в транслитерации, реальная длина оказывается больше: с учётом двойных фамилий и длинных отчеств она может достигать 60–70 символов. Выбрано 100 с запасом. Логин — VARCHAR(30). Логин является произвольной строкой для входа в систему. Выбрано 30 как разумный предел, исключающий избыточно длинные значения.

Таблица 4: Таблица зон парковки

Зона парковки – parking_zone							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	зона_парковки_id	parking_zone_id	SERIAL	-	PK	-	NOT NULL
2	название	name	VARCHAR(60)	-	-	-	NOT NULL
3	тип_зоны_id	zone_type_id	INT	-	FK	zone_typ	NOT NULL
4	район_города	city_district	VARCHAR(50)	-	-	-	NOT NULL
5	макс_авто	max_cars	INT	-	-	-	NOT NULL
6	статус_зоны_id	zone_status_id	INT	-	FK	zone_st	NOT NULL

Обоснование: название зоны — VARCHAR(60). Зоны получают описательные имена вроде «ТЦ Галерея — северный въезд» (29 символов) или «Площадь Восстания — восточная сторона» (38 символов). Выбрано 60 как граница, перекрывающая самые длинные реальные описания. Район города — VARCHAR(50): административные округа Москвы имеют официальные полные названия длиной до 38 символов — например «Северо-Восточный административный округ» (38 символов) и «Юго-Восточный административный округ» (36 символов). Выбрано 50 как граница, перекрывающая самые длинные реальные названия районов с небольшим запасом.

Таблица 5: Справочник типов зон парковки

Тип зоны парковки – parking_zone_type							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	тип_зоны_id	zone_type_id	SERIAL	-	PK	-	NOT NULL
2	название	name	VARCHAR(20)	-	UQ	-	NOT NULL

Обоснование: название типа зоны — VARCHAR(20). Текущие значения: «Бесплатная», «Платная», «Запрещённая» — максимум 11 символов. Перечень типов короткий и устоявшийся, новые значения вряд ли будут длиннее. Выбрано 20 как ближайшая граница выше наблюдаемого максимума.

Таблица 6: Справочник статусов зоны парковки

Статус зоны парковки – parking_zone_status							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	статус_зоны_id	zone_status_id	SERIAL	-	PK	-	NOT NULL
2	название	name	VARCHAR(25)	-	UQ	-	NOT NULL

Обоснование: название статуса зоны — VARCHAR(25). Текущие значения: «Активна», «Временно закрыта», «Недоступна» — максимум 16 символов. Выбрано 25 как ближайшая граница выше наблюдаемого максимума.

Таблица 7: Таблица координат зон парковки

Координаты зоны парковки – parking_zone_point							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	координаты_зоны_id	parking_zone_point_id	SERIAL	-	PK	-	NOT NULL
2	зона_парковки_id	parking_zone_id	INT	-	FK	park_zone	NOT NULL
3	номер_вершины	vertex_number	INT	-	-	-	NOT NULL
4	широта	latitude	NUMERIC(8, 5)	-	-	-	NOT NULL
5	долгота	longitude	NUMERIC(8, 5)	-	-	-	NOT NULL

Таблица 8: Таблица автомобилей

Автомобиль – car							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	авто_id	car_id	SERIAL	-	PK	-	NOT NULL
2	гос_номер	reg_number	VARCHAR(9)	-	UQ	-	NOT NULL
3	марка	brand	VARCHAR(50)	-	-	-	NOT NULL
4	модель	model	VARCHAR(70)	-	-	-	NOT NULL
5	год_выпуска	manufacture_year	INT	-	-	-	NOT NULL
6	vin_номер	vin	VARCHAR(17)	-	UQ	-	NOT NULL
7	статус_id	car_status_id	INT	-	FK	car_status	NOT NULL

Обоснование: гос. номер — VARCHAR(9). По ГОСТ Р 50577–2018 стандартный российский знак имеет формат «A000AA 777» — 9 символов с учётом пробела перед кодом региона. Выбрано 9 с запасом на возможные новые форматы. VIN — VARCHAR(17): стандарт ISO 3779 жёстко фиксирует длину идентификатора ровно в 17 символов. Марка — VARCHAR(50): названия марок автомобилей, присутствующих в российском автопарке, короткие: «BMW» (3), «Volkswagen» (10), «Mercedes-Benz» (13), «Great Wall Motors» (16) — реальный максимум не превышает 20 символов. Выбрано 50 с запасом. Модель — VARCHAR(70): обозначение модели с индексом кузова и модификации обычно не превышает 30 символов, например «Transporter T6.1 2.0 TDI» (24 символа). Встречаются и длинные наименования — «AS55K YAAS EDITION Abu Dhabi Sheikh Sultan Bin Rashed Al Nahyan» (63 символа). Выбрано 70 с запасом.

Таблица 9: Таблица статусов автомобиля

Статус автомобиля – car_status							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	статус_id	car_status_id	SERIAL	-	PK	-	NOT NULL
2	название	name	VARCHAR(25)	-	UQ	-	NOT NULL

Обоснование: название статуса автомобиля — VARCHAR(25). Текущие значения: «Свободен», «В аренде», «На обслуживании», «Неисправен» — максимум 15 символов. Выбрано 25 как ближайшая граница выше наблюдаемого максимума.

Таблица 10: Таблица парковочных сессий

Парковочная сессия – parking_session							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	сессия_id	parking_session_id	SERIAL	-	PK	-	NOT NULL
2	авто_id	car_id	INT	-	FK	car	NOT NULL
3	зона_парковки_id	parking_zone_id	INT	-	FK	park_zone	NOT NULL
4	время_въезда	entry_time	TIMESTAMP	-	-	-	NOT NULL
5	время_выезда	exit_time	TIMESTAMP	NULL	-	-	NULL

Таблица 11: Таблица треков движения

Трек движения – track							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	трек_id	track_id	SERIAL	-	PK	-	NOT NULL
2	авто_id	car_id	INT	-	FK	car	NOT NULL
3	время_начала	start_time	TIMESTAMP	-	-	-	NOT NULL
4	время_конца	end_time	TIMESTAMP	NULL	-	-	NULL
5	статус_трека_id	track_status_id	INT	-	FK	track_status_type	NOT NULL
6	вид_трека_id	track_kind_id	INT	-	FK	track_kind_type	NOT NULL

Таблица 12: Таблица типов статусов трека

Тип статуса трека – track_status_type							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	статус_трека_id	track_status_id	SERIAL	-	PK	-	NOT NULL
2	название_типа	name	VARCHAR(20)	-	-	-	NOT NULL

Обоснование: название статуса трека — VARCHAR(20). Это технический статус записи: «Активный», «Завершён», «Архивный» — максимум 8 символов. Выбрано 20 как граница, при которой можно добавить новый статус, например «В обработке» (12 символов), без изменения схемы.

Таблица 13: Таблица типов видов трека

Тип вида трека – track_kind_type							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	вид_трека_id	track_kind_id	SERIAL	-	PK	-	NOT NULL
2	название_типа	name	VARCHAR(25)	-	-	-	NOT NULL

Обоснование: название вида трека — VARCHAR(25). Текущие значения: «Пользовательский» (16 символов), «Сервисный» (9 символов) — максимум 16 символов. Выбрано 25 как ближайшая граница выше максимума.

Таблица 14: Таблица точек трека движения

Координаты точки трека – track_point							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	id_точки	track_point_id	SERIAL	-	PK	-	NOT NULL
2	id_трека	track_id	INT	-	FK	mov_track	NOT NULL
3	id_авто	car_id	INT	-	FK	car	NULL
4	широта	latitude	NUMERIC(8, 5)	-	-	-	NOT NULL
5	долгота	longitude	NUMERIC(8, 5)	-	-	-	NOT NULL
6	скорость	speed_kmh	NUMERIC(5, 2)	NULL	-	-	NULL
7	источник_id	data_source_id	INT	-	FK	data_source_type	NOT NULL

Таблица 15: Таблица источников данных

Источник данных – data_source_type							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	источник_id	data_source_id	SERIAL	-	PK	-	NOT NULL
2	название	name	VARCHAR(25)	-	UQ	-	NOT NULL

Обоснование: название источника данных — VARCHAR(25). Перечень источников технически закрытый: GPS, ГЛОНАСС, инерциальная навигация и их комбинации. Самое длинное значение «Инерциальная система» — 20 символов. Выбрано 25 как граница вплотную к максимуму.

Таблица 16: Таблица тревожных событий

Тревожное событие – alert_event							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	событие_id	alert_event_id	SERIAL	-	PK	-	NOT NULL
2	авто_id	car_id	INT	-	FK	car	NOT NULL
3	сотрудник_id	employee_id	INT	NULL	FK	employee	NULL
4	тип_события_id	alert_event_type_id	INT	-	FK	alert_event_type	NOT NULL
5	широта	latitude	NUMERIC(8, 5)	NULL	-	-	NULL
6	долгота	longitude	NUMERIC(8, 5)	NULL	-	-	NULL
7	описание	description	TEXT	NULL	-	-	NULL
8	статус_id	status_id	INT	-	FK	event_process_status	NOT NULL

Таблица 17: Таблица статусов обработки события

Статус обработки события – alert_event_process_status							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	статус_id	status_id	SERIAL	-	PK	-	NOT NULL
2	название	name	VARCHAR(20)	-	UQ	-	NOT NULL

Обоснование: название статуса обработки события — VARCHAR(20). Жизненный цикл события — фиксированный: «Новое», «В работе», «Закрыто» — максимум 8 символов. Выбрано 20 с запасом на детализацию, например «Передано диспетчеру» (19 символов).

Таблица 18: Таблица типов тревожных событий

Тип тревожного события – alert_event_type							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	тип_события_id	alert_event_type_id	SERIAL	-	PK	-	NOT NULL
2	название_типа	name	VARCHAR(25)	-	UQ	-	NOT NULL
3	описание	description	TEXT	-	-	-	NULL

Обоснование: название типа тревожного события — VARCHAR(25). Текущие значения: «Потеря связи», «Превышение скорости», «Резкое торможение», «Удар», «Попытка угона», «Выезд за границу» — максимум 19 символов. Названия типов событий должны оставаться краткими метками, поэтому выбрано 25.

Таблица 19: Таблица обращений

Обращение к геомониторингу – geo_request							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	запрос_id	geo_request_id	SERIAL	-	PK	-	NOT NULL
2	сотрудник_id	employee_id	INT	-	FK	employee	NOT NULL
3	авто_id	car_id	INT	-	FK	car	NOT NULL
4	тип_запроса_id	geo_request_type_id	INT	-	FK	geo_request_type	NOT NULL
5	цель_запроса	request_goal	TEXT	-	-	-	NOT NULL

Таблица 20: Таблица типов обращения к геомониторингу

Тип обращения к геомониторингу – geo_request_type							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	тип_запрос_id	geo_request_type_id	SERIAL	-	PK	-	NOT NULL
2	название_типа	name	VARCHAR(25)	-	UQ	-	NOT NULL
3	описание	description	TEXT	-	-	-	NULL

Обоснование: название типа обращения к геомониторингу — VARCHAR(25). Текущие значения: «Текущие координаты» (18 символов), «История событий» (16 символов) — максимум 18 символов. Выбрано 25 как граница вплотную к максимуму.

3.2. Лингвистическое описание схемы базы данных

1. Тип сотрудника (тип_сотрудника_id(SERIAL), название_типа(VARCHAR(30)), описание(TEXT)).
2. Статус сотрудника (статус_id(SERIAL), название(VARCHAR(20))).
3. Сотрудник (сотрудник_id(SERIAL), тип_сотрудника_id(INT), фео(VARCHAR(100)), логин(VARCHAR(30)), статус_id(INT)).
4. Тип зоны парковки (тип_зоны_id(SERIAL), название(VARCHAR(20))).
5. Статус зоны парковки (статус_зоны_id(SERIAL), название(VARCHAR(25))).
6. Зона парковки (зона_парковки_id(SERIAL), название(VARCHAR(60)), тип_зоны_id(район_города(VARCHAR(50)), макс_авто(INT), статус_зоны_id(INT)).
7. Координаты зоны парковки (координаты_зоны_id(SERIAL), зона_парковки_id(INT), номер_вершины(INT), широта(NUMERIC(8, 5)), долгота(NUMERIC(8, 5))).
8. Статус автомобиля (статус_id(SERIAL), название(VARCHAR(25))).
9. Автомобиль (авто_id(SERIAL), гос_номер(VARCHAR(9)), марка(VARCHAR(50)), модель(VARCHAR(70)), год_выпуска(INT), vin_номер(VARCHAR(17)), статус_id(INT)).

10. Парковочная сессия (сессия_id(SERIAL), авто_id(INT), зона_парковки_id(INT), время_въезда(TIMESTAMP), время_выезда(TIMESTAMP)).
11. Тип статуса трека (статус_трека_id(SERIAL), название_типа(VARCHAR(20))).
12. Тип вида трека (вид_трека_id(SERIAL), название_типа(VARCHAR(25))).
13. Трек движения (трек_id(SERIAL), авто_id(INT), время_начала(TIMESTAMP), время_конца(TIMESTAMP), статус_трека_id(INT), вид_трека_id(INT)).
14. Источник данных (источник_id(SERIAL), название(VARCHAR(25))).
15. Координаты точки трека (id_точки(SERIAL), id_трека(INT), id_авто(INT), широта(NUMERIC(8, 5)), долгота(NUMERIC(8, 5)), скорость(NUMERIC(5, 2)), источник_id(INT)).
16. Тип тревожного события (тип_события_id(SERIAL), название_типа(VARCHAR(25)), описание(TEXT)).
17. Статус обработки события (статус_id(SERIAL), название(VARCHAR(20))).
18. Тревожное событие (событие_id(SERIAL), авто_id(INT), сотрудник_id(INT), тип_события_id(INT), широта(NUMERIC(8, 5)), долгота(NUMERIC(8, 5)), описание(TEXT), статус_id(INT)).
19. Тип обращения к геомониторингу (тип_запрос_id(SERIAL), название_типа(VARCHAR(25)), описание(TEXT)).
20. Обращение к геомониторингу (запрос_id(SERIAL), сотрудник_id(INT), авто_id(INT), тип_запроса_id(INT), цель_запроса(TEXT)).

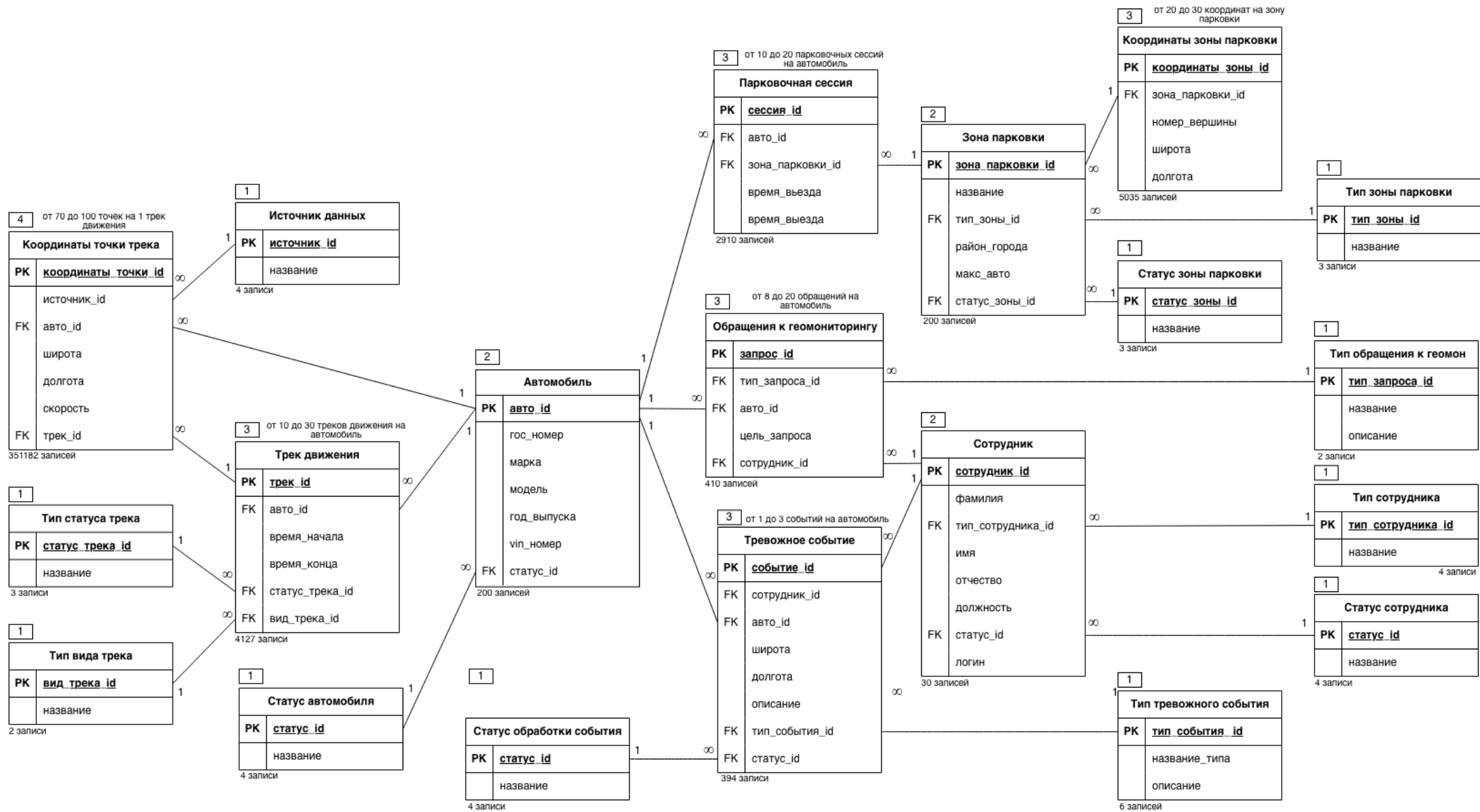


Рис. 3: Графическое описание схемы базы данных на русском языке

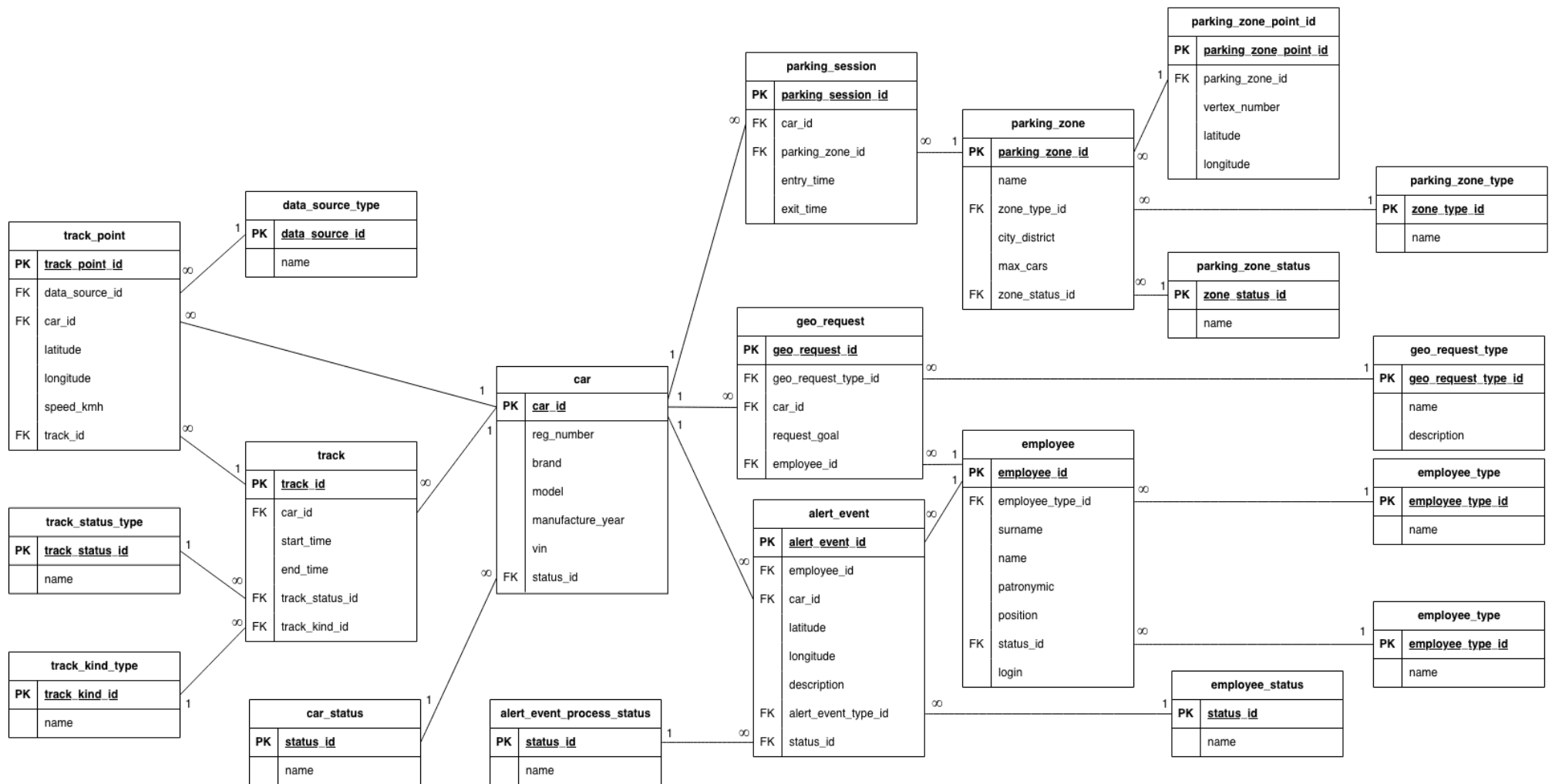


Рис. 4: Графическое описание схемы базы данных на английском языке

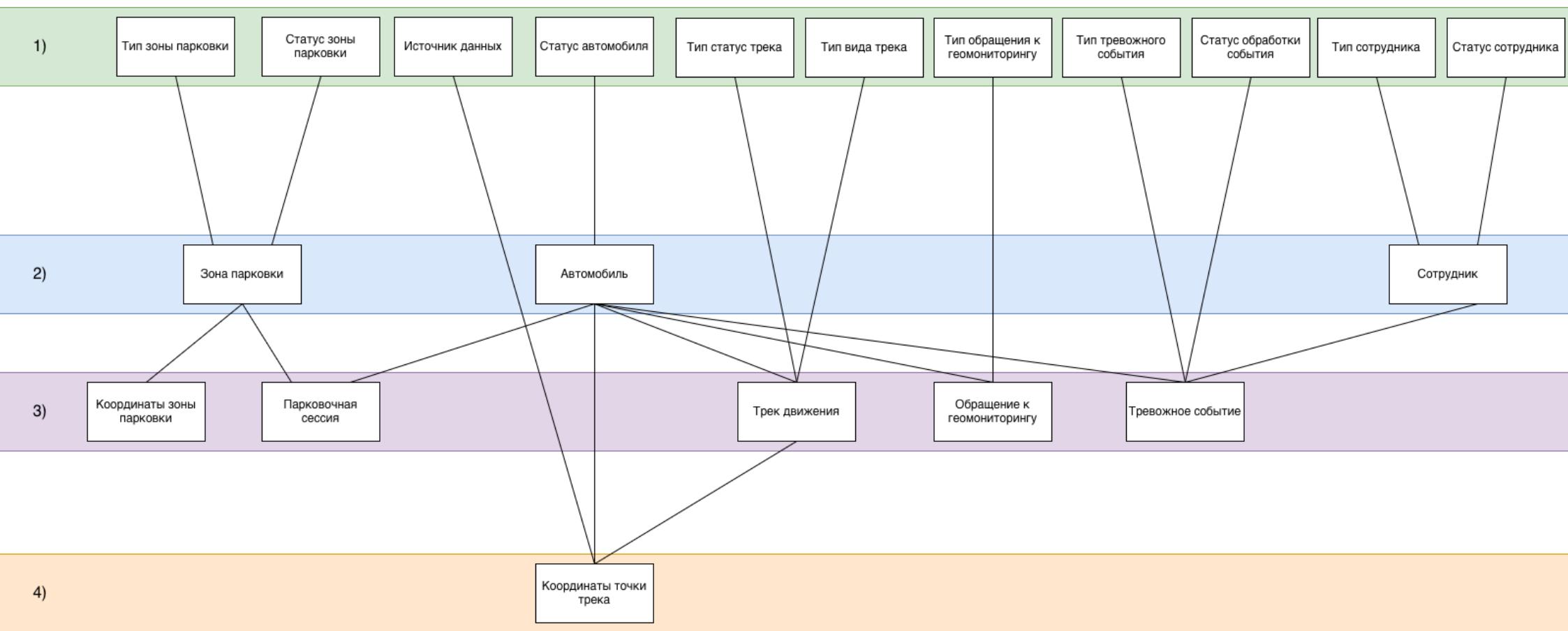


Рис. 5: Схема заполнения базы данных

3.3. Исходный текст программы генерации схемы базы данных

База данных создана средствами СУБД PostgreSQL с использованием SQL-скриптов, приведенных в [приложении А](#) (схема на рис. 3).

3.4. Исходный текст программы заполнения базы данных

База данных заполнена тестовыми данными с помощью модуля `rgx`, на языке Golang. Исходный код генератора тестовых данных приведён в [приложении В](#) (схема на рис. 4).

Лист проверки выполнения запросов
курсовой работы по дисциплине
«Теоретические основы баз данных» 4 семестр

Имя, отчество, фамилия Абдуатуллин Тимур Ринатович Группа 5130201/4000_3

Тема работы Реализация Базы данных для предметной области «Геомониторинг автопарка каршеринга»

№ п.п.	текст запроса	принято	
		запрос	отчет
1.1	Найти все автомобили, которые парковались в зоне парк А, у которого тревож.соб В, и источник координат устройства С.	+	+
2	Для зоны А, посчитать число авто, которые парковались в зоне парковки А.	+	+
3	Для каждого сотрудника посчитать, число тревож. соб и кол-во автомобилей. Построить 2-д гистограмму.	+	+
4	а) Найти зоны парков в которых парк имеет кол-во авто. б) Найти минимальное.....	+	+
5	Найти число зон парковок, соединяемых числом парковавшихся автомобилей. Построить 2-д гистограмму.	+	+
6	Найти сотрудников, которые обр. больше тревож. соб, чем сотрудник А.	+	+
7	Найти зоны парковки, в которых никогда не парковался автомобиль А.	+	+
8	Для каждого источника данных, посчитать кол-во координат. ↑ и автомобиля Построить 3-д гистограмму.	+	+
9	Для всех тревог авто А, поменять статус трега с А на Б.	+	+
10		+	+

1.2:.... Не менее 3 раз.

4. Программирование

4.1. Запрос 1.1

Найти все автомобили, которые парковались в типе зоны парковки "А", у которого тревожное событие "В" и источник координат "С".

```
1 SELECT DISTINCT c.car_id, c.reg_number, c.brand, c.model
2 FROM car c
3 JOIN parking_session ps ON ps.car_id = c.car_id
4 JOIN parking_zone pz ON pz.parking_zone_id = ps.parking_zone_id
5 JOIN parking_zone_type pzt ON pzt.zone_type_id = pz.zone_type_id
6 JOIN alert_event ae ON ae.car_id = c.car_id
7 JOIN alert_event_type aet ON aet.alert_event_type_id =
   ae.alert_event_type_id
8 JOIN track t ON t.car_id = c.car_id
9 JOIN track_point tp ON tp.track_id = t.track_id
10 JOIN data_source_type dst ON dst.data_source_id = tp.data_source_id
11 WHERE pzt.name = 'Бесплатная'
12 AND aet.name = 'Потеря связи'
13 AND dst.name = 'GPS';
```

Listing 1: Запрос 1.1

car_id	reg_number	brand	model
5	A005AA81	Volkswagen	Polo
6	A006AA77	Volkswagen	Passat
9	A009AA80	Skoda	Superb
11	A011AA77	Volkswagen	Touareg
14	A014AA80	Mazda	MX-5
17	A017AA78	Toyota	Land Cruiser
25	A025AA81	Mazda	CX-9
26	A026AA77	Skoda	Karoq
29	A029AA80	Toyota	RAV4
38	A038AA79	Volkswagen	Taos
44	A044AA80	BMW	X5
46	A046AA77	Volkswagen	Taos
49	A049AA80	BMW	7 Series
51	A051AA77	Skoda	Octavia
52	A052AA78	Nissan	X-Trail
59	A059AA80	Skoda	Kodiaq
61	A061AA77	Nissan	Almera
67	A067AA78	BMW	5 Series
68	A068AA79	Nissan	X-Trail
74	A074AA80	Skoda	Karoq
77	A077AA78	Renault	Arkana

Рис. 6: Результат выполнения запроса 1.1

Время выполнения запроса:

Planning Time: 3.444 ms

Execution Time: 148.770 ms

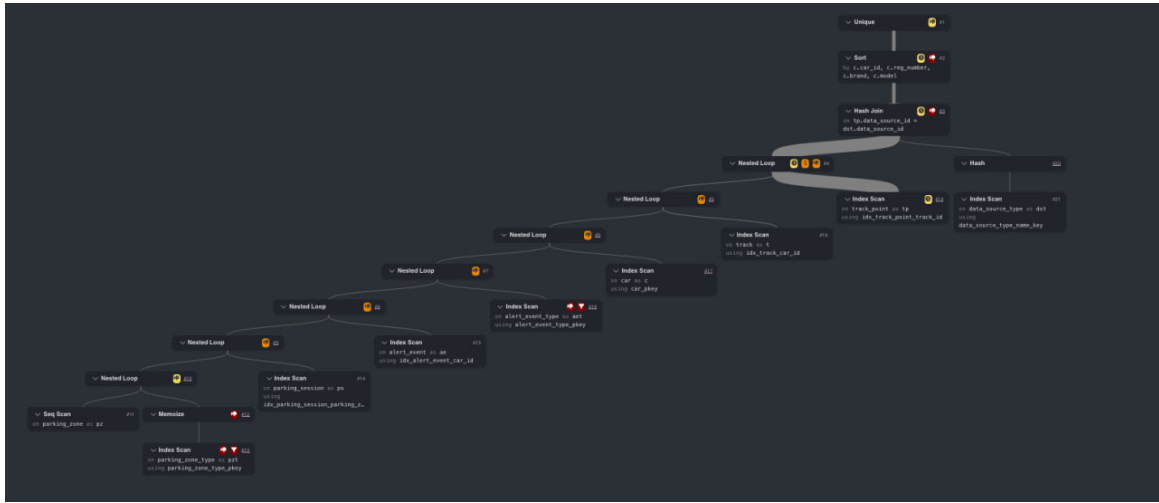


Рис. 7: Последовательность выполнения запроса 1.1

Сначала база данных последовательно читает таблицу **parking_zone** и для каждой зоны через кэшированный поиск по индексу проверяет её тип, оставляя только тип «Бесплатная». По найденным зонам с помощью индекса находятся парковочные сессии. Для каждой сессии по **car_id** ищутся тревожные события, и тип каждого события проверяется — остаются только «Потеря связи». Затем по индексу достаются данные автомобиля, после чего находятся его треки и все точки этих треков. Точки трека соединяются с источниками данных через Hash Join, и остаются только точки с источником «GPS». В самом конце результат сортируется и убираются дубликаты.

Реляционная формула запроса:

$$\pi_{\text{car_id, reg_number, brand, model}} \left(\sigma_{\text{pzt.name}=A \wedge \text{aet.name}=B \wedge \text{dst.name}=C} \left(\begin{array}{l} \text{car} \\ \bowtie_{\text{car_id}} \text{parking_session} \\ \bowtie_{\text{parking_zone_id}} \text{parking_zone} \\ \bowtie_{\text{zone_type_id}} \text{parking_zone_type} \\ \bowtie_{\text{car_id}} \text{alert_event} \\ \bowtie_{\text{alert_event_type_id}} \text{alert_event_type} \\ \bowtie_{\text{car_id}} \text{track} \\ \bowtie_{\text{track_id}} \text{track_point} \\ \bowtie_{\text{data_source_id}} \text{data_source_type} \end{array} \right) \right)$$

4.2. Запрос 1.2

Найти все автомобили, которые парковались в типе зоны парковки "А", у которого тревожное событие "В" и источник координат "С", не менее 3 раз.

```
1 SELECT c.car_id, c.reg_number, c.brand, c.model,  
2       COUNT(DISTINCT ps.parking_session_id) AS cnt  
3 FROM car c  
4 JOIN parking_session ps ON ps.car_id = c.car_id  
5 JOIN parking_zone pz ON pz.parking_zone_id = ps.parking_zone_id  
6 JOIN parking_zone_type pzt ON pzt.zone_type_id = pz.zone_type_id  
7 JOIN alert_event ae ON ae.car_id = c.car_id  
8 JOIN alert_event_type aet ON aet.alert_event_type_id =  
   ae.alert_event_type_id  
9 JOIN track t ON t.car_id = c.car_id  
10 JOIN track_point tp ON tp.track_id = t.track_id  
11 JOIN data_source_type dst ON dst.data_source_id = tp.data_source_id  
12 WHERE pzt.name = 'Бесплатная'  
13    AND aet.name = 'Потеря связи'  
14    AND dst.name = 'GPS'  
15 GROUP BY c.car_id, c.reg_number, c.brand, c.model  
16 HAVING COUNT(DISTINCT ps.parking_session_id) >= 3;
```

Listing 2: Запрос 1.2

car_id	reg_number	brand	model	cnt
5	A005AA81	Volkswagen	Polo	5
6	A006AA77	Volkswagen	Passat	5
9	A009AA80	Skoda	Superb	7
11	A011AA77	Volkswagen	Touareg	4
17	A017AA78	Toyota	Land Cruiser	5
25	A025AA81	Mazda	CX-9	7
26	A026AA77	Skoda	Karoq	6
38	A038AA79	Volkswagen	Taos	6
46	A046AA77	Volkswagen	Taos	3
49	A049AA80	BMW	7 Series	4
51	A051AA77	Skoda	Octavia	5
52	A052AA78	Nissan	X-Trail	6
59	A059AA80	Skoda	Kodiaq	6
61	A061AA77	Nissan	Almera	7
67	A067AA78	BMW	5 Series	5
68	A068AA79	Nissan	X-Trail	4
78	A078AA79	BMW	X5	10
79	A079AA80	BMW	7 Series	6
85	A085AA81	Kia	Sportage	5
101	A101AA77	Nissan	X-Trail	5
106	A106AA77	Mazda	3	5

Рис. 8: Результат выполнения запроса 1.2

Время выполнения запроса:

Planning Time: 2.730 ms

Execution Time: 113.795 ms

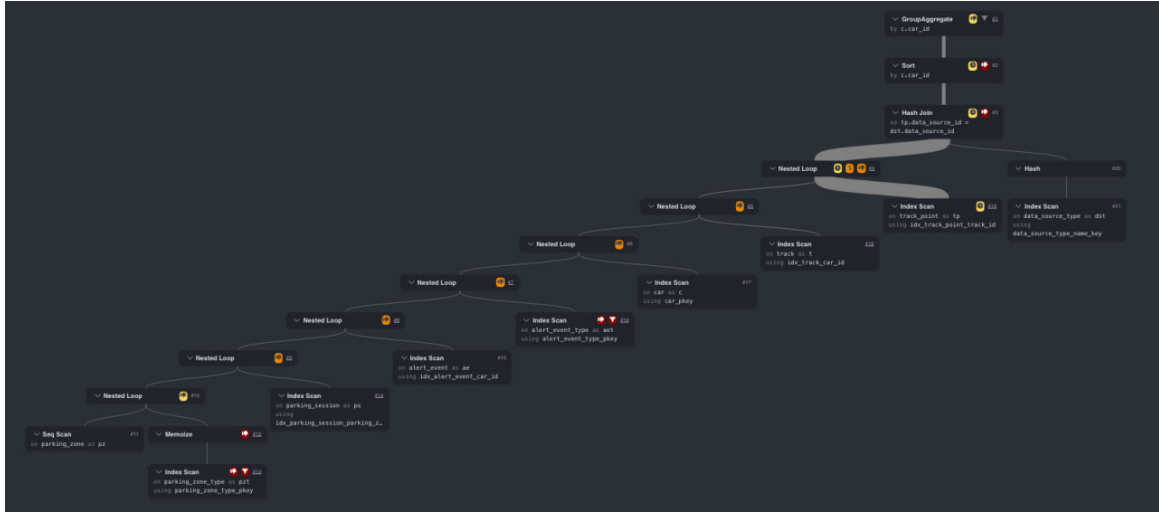


Рис. 9: Последовательность выполнения запроса 1.2

Последовательность выполнения полностью совпадает с запросом 1.1. Единственное отличие: после соединения всех таблиц из-за механики **DISTINCT** данные сортируются по **car_id**, а затем GroupAggregate считает количество уникальных парковочных сессий для каждого автомобиля. Автомобили, у которых таких сессий меньше трёх, отфильтровываются.

Реляционная формула запроса:

$$\begin{aligned}
 & \gamma_{\text{car_id, reg_number, brand, model, cnt}} \left(\right. \\
 & \quad \sigma_{\text{pzt.name}=A \wedge \text{aet.name}=B \wedge \text{dst.name}=C \wedge \text{COUNT}(\text{parking_session_id}) \geq 3} \left(\right. \\
 & \quad \quad \text{car} \\
 & \quad \quad \bowtie_{\text{car_id}} \text{parking_session} \\
 & \quad \quad \bowtie_{\text{parking_zone_id}} \text{parking_zone} \\
 & \quad \quad \bowtie_{\text{zone_type_id}} \text{parking_zone_type} \\
 & \quad \quad \bowtie_{\text{car_id}} \text{alert_event} \\
 & \quad \quad \bowtie_{\text{alert_event_type_id}} \text{alert_event_type} \\
 & \quad \quad \bowtie_{\text{car_id}} \text{track} \\
 & \quad \quad \bowtie_{\text{track_id}} \text{track_point} \\
 & \quad \quad \bowtie_{\text{data_source_id}} \text{data_source_type} \\
 & \quad \left. \right) \\
 & \left. \right)
 \end{aligned}$$

4.3. Запрос 2

Для сотрудника "А", который обрабатывал тревожные события, посчитать число автомобилей, которые парковались в зоне парковки "В".

```
1 SELECT COUNT(DISTINCT ps.car_id) AS car_count
2 FROM parking_session ps
3 JOIN parking_zone pz ON pz.parking_zone_id = ps.parking_zone_id
4 JOIN alert_event ae ON ae.car_id = ps.car_id
5 JOIN employee e ON e.employee_id = ae.employee_id
6 WHERE pz.name = 'ТЦ Мега - парковка В'
7 AND e.full_name = 'Иванов Игорь Николаевич';
```

Listing 3: Запрос 2



car_count

2

Рис. 10: Результат выполнения запроса 2

Время выполнения запроса:

Planning Time: 0.553 ms

Execution Time: 0.919 ms

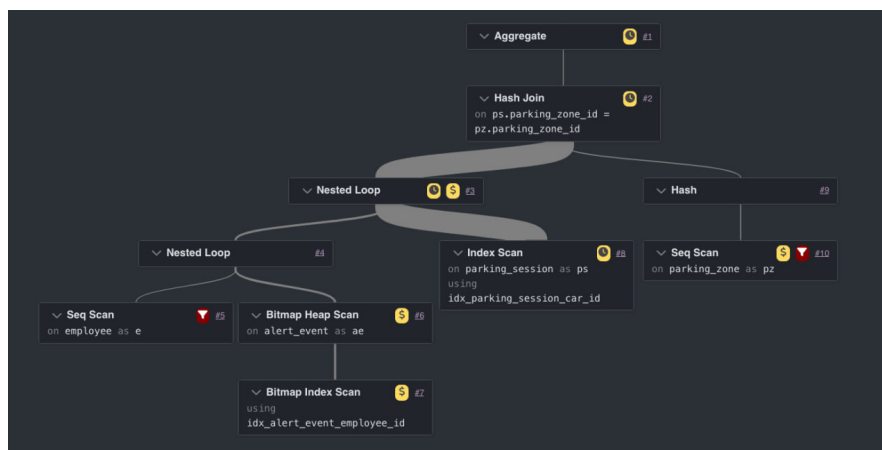


Рис. 11: Последовательность выполнения запроса 2

Сначала последовательно читается таблица **employee** и находится нужный сотрудник. По его **employee_id** через битовый индекс находятся все тревожные события, которые он обработал. Для каждого события по индексу ищутся парковочные сессии того же автомобиля. Затем через Hash Join проверяется, относится ли сессия к нужной зоне парковки. В конце агрегат считает количество уникальных автомобилей.

Реляционная формула запроса:

$$\pi_{\text{COUNT}(\text{car_id}) \rightarrow \text{car_count}} \left(\sigma_{\text{pz.name} = B \wedge \text{full_name} = A} \left(\begin{array}{l} \text{parking_session} \\ \bowtie_{\text{parking_zone_id}} \text{parking_zone} \\ \bowtie_{\text{car_id}} \text{alert_event} \\ \bowtie_{\text{employee_id}} \text{employee} \end{array} \right) \right)$$

4.4. Запрос 3

Для каждого сотрудника посчитать число тревожных событий и количество автомобилей.

```
1 SELECT e.employee_id, e.full_name,  
2       COUNT(DISTINCT ae.alert_event_id) AS alert_count,  
3       COUNT(DISTINCT ae.car_id) AS car_count  
4 FROM employee e  
5 LEFT JOIN alert_event ae ON ae.employee_id = e.employee_id  
6 GROUP BY e.employee_id, e.full_name  
7 ORDER BY e.employee_id;
```

Listing 4: Запрос 3

employee_id	full_name	alert_count	car_count
1	Иванов Игорь Николаевич	15	15
2	Смирнов Иван Максимович	9	9
3	Кузнецов Михаил Николаевич	11	11
4	Попов Игорь Сергеевич	11	11
5	Васильев Павел Васильевич	18	18
6	Петров Кирилл Васильевич	16	15
7	Соколов Роман Александрович	15	15
8	Михайлов Алексей Павлович	15	14
9	Новиков Василий Андреевич	10	10
10	Фёдоров Николай Дмитриевич	15	15
11	Морозов Алексей Александрович	8	8
12	Волков Василий Константинович	12	12
13	Алексеев Константин Васильевич	11	11
14	Лебедев Виктор Андреевич	11	11
15	Семёнов Артём Артёмович	18	18
16	Егоров Кирилл Максимович	15	15
17	Павлов Игорь Романович	13	13
18	Козлов Николай Николаевич	11	10
19	Степанов Кирилл Григорьевич	21	20
20	Николаев Артём Григорьевич	13	12
21	Орлов Максим Павлович	10	10

Рис. 12: Результат выполнения запроса 3

Время выполнения запроса:

Planning Time: 0.288 ms

Execution Time: 0.297 ms

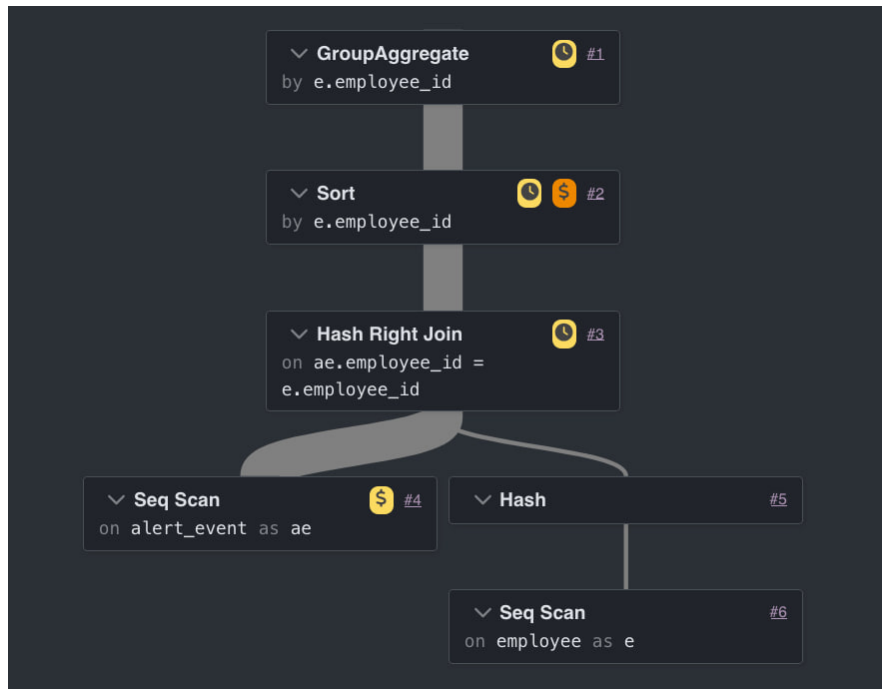


Рис. 13: Последовательность выполнения запроса 3

Обе таблицы — **employee** и **alert_event** — читаются последовательно, после чего соединяются через Hash Right Join. Такой тип соединения нужен, чтобы в результат попали даже те сотрудники, у которых нет ни одного тревожного события. Затем данные сортируются по **employee_id**, и GroupAggregate считает для каждого сотрудника количество уникальных событий и уникальных автомобилей.

Реляционная формула запроса:

$$\begin{aligned}
 &\pi_{\text{employee_id}, \text{full_name}, \text{alert_count}, \text{car_count}} \left(\right. \\
 &\quad \gamma_{\text{employee_id}, \text{full_name}; \text{COUNT}(\text{alert_event_id}) \rightarrow \text{alert_count}, \text{COUNT}(\text{car_id}) \rightarrow \text{car_count}} \left(\right. \\
 &\quad \quad \text{employee} \bowtie (\text{left})_{\text{employee_id}} \text{alert_event} \\
 &\quad \left. \right) \\
 &\left. \right)
 \end{aligned}$$

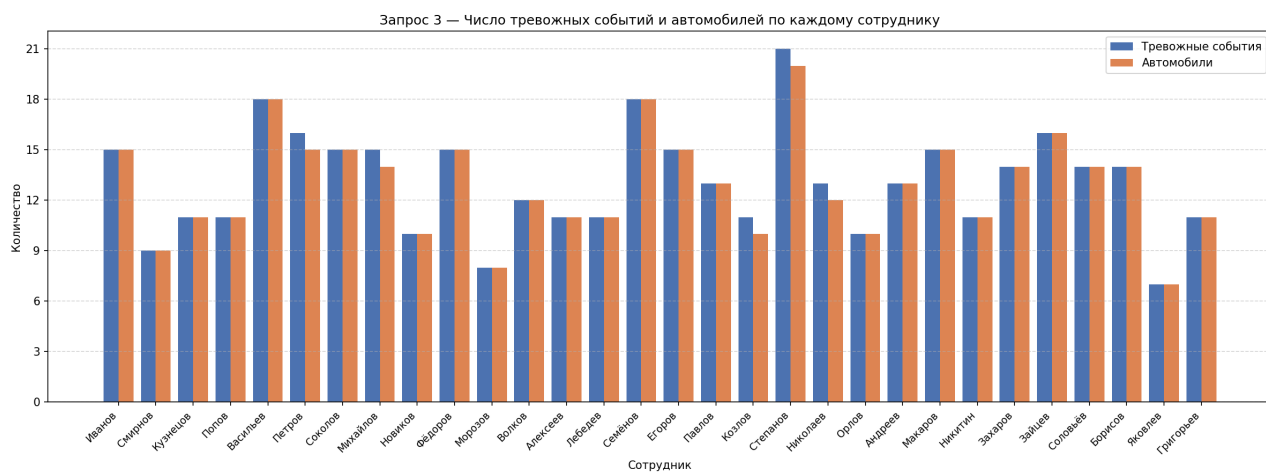


Рис. 14: Гистограмма запроса 3 — число тревожных событий и автомобилей по каждому сотруднику

4.5. Запрос 4

- Найти зоны парковки в которых парковалось максимальное количество автомобилей.
- Найти зоны парковки в которых парковалось минимальное количество автомобилей.

```
1  -- MAX
2  SELECT pz.parking_zone_id, pz.name, COUNT(DISTINCT ps.car_id) AS
   cnt
3  FROM parking_zone pz
4  JOIN parking_session ps ON ps.parking_zone_id = pz.parking_zone_id
5  GROUP BY pz.parking_zone_id, pz.name
6  HAVING COUNT(DISTINCT ps.car_id) = (
7      SELECT MAX(sub.car_count)
8      FROM (
9          SELECT COUNT(DISTINCT ps2.car_id) AS car_count
10         FROM parking_session ps2
11         GROUP BY ps2.parking_zone_id
12     ) sub
13 );
14
15 -- MIN
16 SELECT pz.parking_zone_id, pz.name, COUNT(DISTINCT ps.car_id) AS
   cnt
17 FROM parking_zone pz
18 JOIN parking_session ps ON ps.parking_zone_id = pz.parking_zone_id
19 GROUP BY pz.parking_zone_id, pz.name
20 HAVING COUNT(DISTINCT ps.car_id) = (
21     SELECT MIN(sub.car_count)
22     FROM (
23         SELECT COUNT(DISTINCT ps2.car_id) AS car_count
24         FROM parking_session ps2
25         GROUP BY ps2.parking_zone_id
26     ) sub
27 );
```

Listing 5: Запрос 4 min/max

parking_zone_id	name	cnt
30	МЦД Одинцово – главный вход	23
32	ТЦ Галерея – боковой вход	23
164	ТЦ Галерея – боковой вход	23

Рис. 15: Результат выполнения запроса 4 (макс)

Время выполнения запроса (макс):

Planning Time: 0.419 ms

Execution Time: 1.743 ms

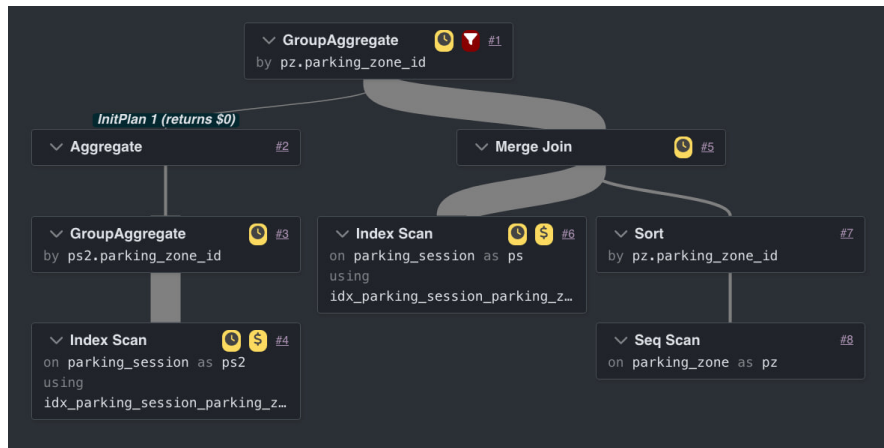


Рис. 16: Последовательность выполнения запроса 4 (макс)

Сначала выполняется подзапрос: по индексу читаются все парковочные сессии, группируются по зоне парковки, и для каждой зоны считается количество уникальных автомобилей. Из этих чисел берётся максимум. Затем основной запрос соединяет **parking_session** и **parking_zone** через Merge Join, группирует результат по зоне и оставляет только те зоны, где количество автомобилей равно найденному максимуму.

Реляционная формула запроса (MAX).

Пусть $M = \gamma_{\text{MAX}(\text{car_count})} \left(\gamma_{\text{parking_zone_id}; \text{COUNT}(\text{car_id}) \rightarrow \text{car_count}}(\text{parking_session}) \right)$.

$$\pi_{\text{parking_zone_id}, \text{name}, \text{cnt}} \left(\sigma_{\text{cnt}=M} \left(\gamma_{\text{parking_zone_id}, \text{name}; \text{COUNT}(\text{car_id}) \rightarrow \text{cnt}} \left(\text{parking_zone} \bowtie_{\text{parking_zone_id}} \text{parking_session} \right) \right) \right)$$

parking_zone_id	name	cnt
144	ТЦ Европейский – подземная парковка	4

Рис. 17: Результат выполнения запроса 4 (мин)

Время выполнения запроса (мин):

Planning Time: 0.335 ms

Execution Time: 1.449 ms

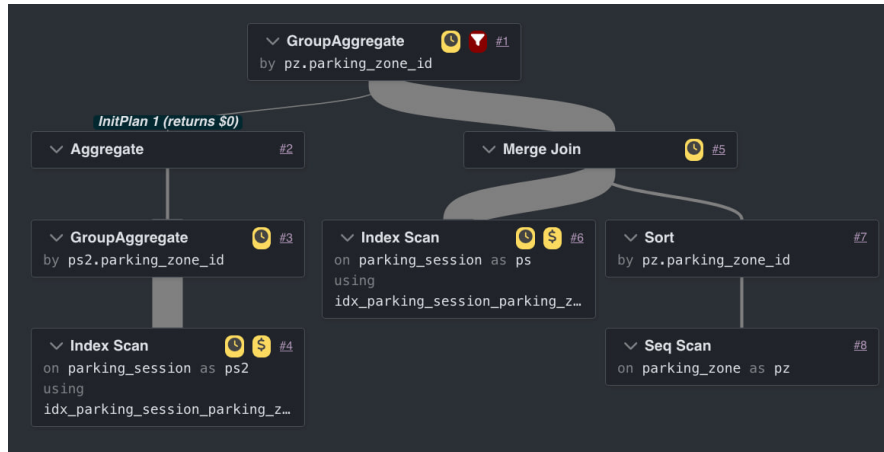


Рис. 18: Последовательность выполнения запроса 4 (мин)

Выполнение полностью аналогично запросу с максимумом. Единственное отличие: подзапрос вычисляет минимальное количество автомобилей среди всех зон, и затем основной запрос оставляет только те зоны, где это количество совпадает с минимумом.

Реляционная формула запроса (MIN).

Пусть $m = \gamma_{\text{MIN}(\text{car_count})} \left(\gamma_{\text{parking_zone_id}; \text{COUNT}(\text{car_id}) \rightarrow \text{car_count}}(\text{parking_session}) \right)$.

$$\begin{aligned}
 & \pi_{\text{parking_zone_id}, \text{name}, \text{cnt}} \left(\right. \\
 & \quad \sigma_{\text{cnt}=m} \left(\right. \\
 & \quad \quad \gamma_{\text{parking_zone_id}, \text{name}; \text{COUNT}(\text{car_id}) \rightarrow \text{cnt}} \left(\right. \\
 & \quad \quad \quad \text{parking_zone} \bowtie_{\text{parking_zone_id}} \text{parking_session} \\
 & \quad \quad \left. \right) \\
 & \quad \left. \right) \\
 & \left. \right)
 \end{aligned}$$

4.6. Запрос 5

Найти число зон парковок, с одинаковым числом паркововавшихся автомобилей для каждого сотрудника.

```
1 SELECT e.full_name, sub.car_count, COUNT(*) AS zone_count
2 FROM employee e
3 JOIN (
4     SELECT ae.employee_id, ps.parking_zone_id,
5           COUNT(DISTINCT ae.car_id) AS car_count
6     FROM alert_event ae
7     JOIN parking_session ps ON ps.car_id = ae.car_id
8     GROUP BY ae.employee_id, ps.parking_zone_id
9 ) sub ON sub.employee_id = e.employee_id
10 GROUP BY e.full_name, sub.car_count
11 ORDER BY e.full_name, sub.car_count;
```

Listing 6: Запрос 5

full_name	car_count	zone_count
Алексеев Константин Васильевич	1	75
Алексеев Константин Васильевич	2	29
Алексеев Константин Васильевич	3	9
Андреев Сергей Игоревич	1	56
Андреев Сергей Игоревич	2	32
Андреев Сергей Игоревич	3	13
Андреев Сергей Игоревич	4	5
Борисов Игорь Максимович	1	79
Борисов Игорь Максимович	2	37
Борисов Игорь Максимович	3	14
Васильев Павел Васильевич	1	94
Васильев Павел Васильевич	2	45
Васильев Павел Васильевич	3	16
Васильев Павел Васильевич	4	3
Волков Василий Константинович	1	81
Волков Василий Константинович	2	26
Волков Василий Константинович	3	8
Волков Василий Константинович	4	2
Григорьев Александр Дмитриевич	1	71
Григорьев Александр Дмитриевич	2	32
Григорьев Александр Дмитриевич	3	7

Рис. 19: Результат выполнения запроса 5

Время выполнения запроса:

Planning Time: 0.437 ms

Execution Time: 4.812 ms

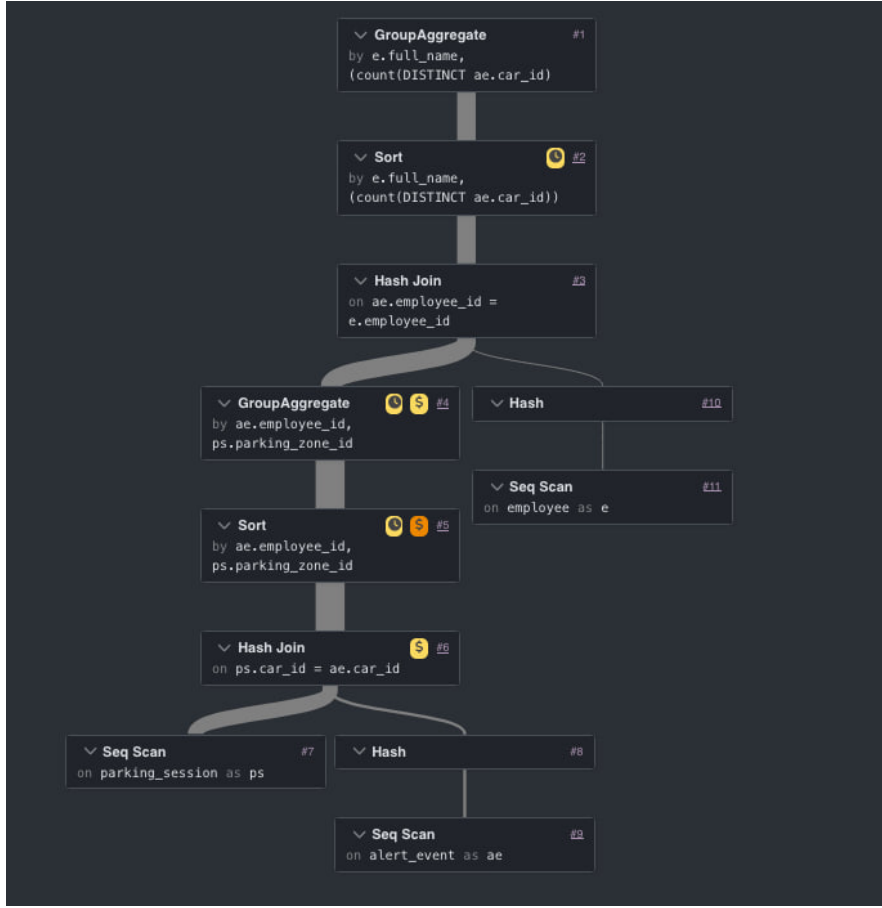


Рис. 20: Последовательность выполнения запроса 5

Сначала читаются таблицы **alert_event** и **parking_session**, которые соединяются по **car_id** через Hash Join. Результат сортируется и группируется по паре (**employee_id**, **parking_zone_id**): для каждой такой пары считается количество уникальных автомобилей — это подзапрос. Затем подзапрос соединяется с **employee** через Hash Join. Итоговые данные снова сортируются и группируются: теперь уже по (**full_name**, **car_count**), чтобы посчитать, сколько зон у сотрудника имеют одинаковое число автомобилей.

Реляционная формула запроса.

Пусть $T = \gamma_{\text{employee_id, parking_zone_id; COUNT(car_id) \rightarrow \text{car_count}}}(\text{alert_event} \bowtie_{\text{car_id}} \text{parking_session})$

$$\begin{aligned}
 & \pi_{\text{full_name, car_count, zone_count}} \left(\right. \\
 & \quad \gamma_{\text{full_name, car_count; COUNT(*) \rightarrow \text{zone_count}}} \left(\right. \\
 & \quad \quad \text{employee} \bowtie_{\text{employee_id}} T \\
 & \quad \left. \right) \\
 & \left. \right)
 \end{aligned}$$

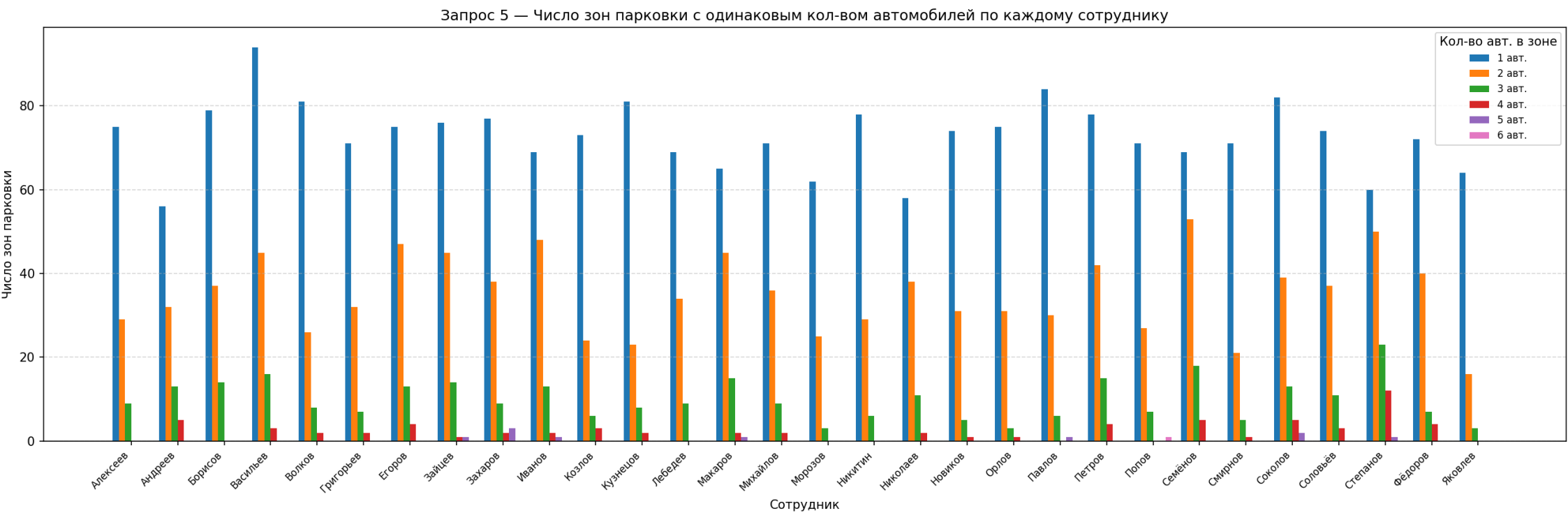


Рис. 21: Гистограмма запроса 5 — число зон парковки с одинаковым количеством автомобилей по каждому сотруднику

4.7. Запрос 6

Найти сотрудников, которые обработали больше тревожных событий, чем сотрудник "А".

```
1 SELECT e.employee_id, e.full_name, COUNT(ae.alert_event_id) AS cnt
2 FROM employee e
3 JOIN alert_event ae ON ae.employee_id = e.employee_id
4 GROUP BY e.employee_id, e.full_name
5 HAVING COUNT(ae.alert_event_id) > (
6     SELECT COUNT(ae2.alert_event_id)
7     FROM alert_event ae2
8     JOIN employee e2 ON e2.employee_id = ae2.employee_id
9     WHERE e2.full_name = 'Иванов Игорь Николаевич'
10 )
11 ORDER BY cnt DESC;
```

Listing 7: Запрос 6

employee_id	full_name	cnt
19	Степанов Кирилл Григорьевич	21
15	Семёнов Артём Артёмович	18
5	Васильев Павел Васильевич	18
6	Петров Кирилл Васильевич	16
26	Зайцев Кирилл Александрович	16

Рис. 22: Результат выполнения запроса 6

Время выполнения запроса:

Planning Time: 0.331 ms

Execution Time: 0.217 ms

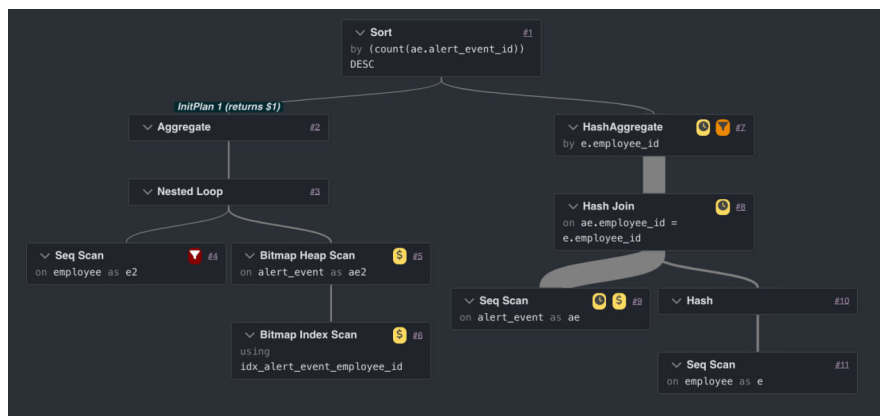


Рис. 23: Последовательность выполнения запроса 6

Сначала выполняется подзапрос: последовательно находится сотрудник А, через битовый индекс находятся все его тревожные события, и агрегат считает их

количество. Затем основной запрос читает все события и всех сотрудников, соединяет их через Hash Join и группирует по каждому сотруднику. Оставляются только те сотрудники, у которых событий больше, чем у A . В конце результат сортируется по убыванию количества событий.

Реляционная формула запроса.

Пусть $C_A = \gamma_{\text{COUNT}(\text{alert_event_id})}(\sigma_{\text{full_name}=A}(\text{employee} \bowtie_{\text{employee_id}} \text{alert_event}))$.

$$\pi_{\text{employee_id}, \text{full_name}, \text{cnt}} \left(\sigma_{\text{cnt} > C_A} \left(\gamma_{\text{employee_id}, \text{full_name}; \text{COUNT}(\text{alert_event_id}) \rightarrow \text{cnt}} \left(\text{employee} \bowtie_{\text{employee_id}} \text{alert_event} \right) \right) \right)$$

4.8. Запрос 7

Найти зоны парковки, в которых никогда не парковался автомобиль "А".

```
1 SELECT pz.parking_zone_id, pz.name
2 FROM parking_zone pz
3 WHERE pz.parking_zone_id NOT IN (
4     SELECT ps.parking_zone_id
5     FROM parking_session ps
6     JOIN car c ON c.car_id = ps.car_id
7     WHERE c.reg_number = 'AAA00177'
8 );
```

Listing 8: Запрос 7

parking_zone_id	name
1	ТЦ Мега – парковка В
2	ВДНХ – северный въезд
3	МЦД Одинцово – уровень 2
4	ЖК Северный – северный въезд
5	ТЦ Рио – уровень 2
6	ТЦ Ереван Плаза – парковка А
7	ТЦ Галерея – главный вход
8	ТЦ Мега – западный въезд
9	МЦД Одинцово – уровень 1
10	Лужники – парковка В
11	Красная площадь – уровень 2
12	ТЦ Авиапарк – уровень 1
13	Красная площадь – боковой вход
14	ТЦ Атриум – северный въезд
15	МЦД Одинцово – уровень 2
16	МЦД Одинцово – уровень 1
17	Коломенское – уровень 1
18	Коломенское – парковка А
19	ТЦ Ереван Плаза – парковка Б
20	ТЦ Авиапарк – южный въезд
21	Измайловский парк – парковка Б

Рис. 24: Результат выполнения запроса 7

Время выполнения запроса:

Planning Time: 0.403 ms

Execution Time: 0.100 ms

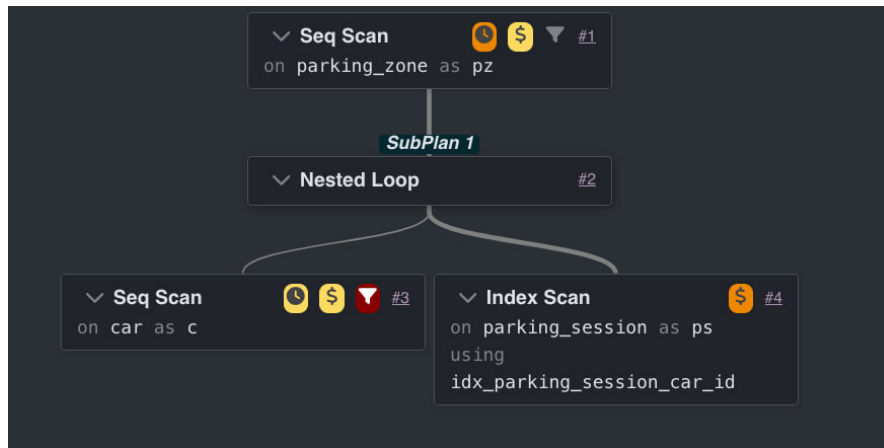


Рис. 25: Последовательность выполнения запроса 7

Сначала выполняется подзапрос: последовательно читается таблица **car**, находится нужный автомобиль по номеру, и по индексу находятся все зоны парковки, где он бывал. Этот список зон хешируется. Затем последовательно читается вся таблица **parking_zone**, и каждая зона проверяется: если она есть в списке — она пропускается. В результат попадают только те зоны, которых в списке нет.

Реляционная формула запроса:

$$\begin{aligned}
 & \pi_{\text{parking_zone_id, name}}(\text{parking_zone}) \\
 & \setminus \\
 & \pi_{\text{parking_zone_id, name}} \left(\right. \\
 & \quad \sigma_{\text{reg_number}=\text{A}} \left(\right. \\
 & \quad \quad \text{car} \\
 & \quad \quad \bowtie_{\text{car_id}} \text{parking_session} \\
 & \quad \quad \bowtie_{\text{parking_zone_id}} \text{parking_zone} \\
 & \quad \left. \right) \\
 & \left. \right)
 \end{aligned}$$

4.9. Запрос 8

Для каждого источника данных, автомобилей, посчитать количество координат.

```
1 SELECT c.car_id, c.reg_number, dst.name AS data_source,  
2        COUNT(tp.track_point_id) AS coord_count  
3 FROM car c  
4 CROSS JOIN data_source_type dst  
5 LEFT JOIN track_point tp  
6     ON tp.car_id = c.car_id  
7     AND tp.data_source_id = dst.data_source_id  
8 GROUP BY c.car_id, c.reg_number, dst.data_source_id, dst.name  
9 ORDER BY c.car_id, dst.name;
```

Listing 9: Запрос 8

car_id	reg_number	data_source	coord_count
1	A001AA77	GPS	284
1	A001AA77	GPS/ГЛОНАСС	277
1	A001AA77	ГЛОНАСС	276
1	A001AA77	Инерциальная система	259
2	A002AA78	GPS	604
2	A002AA78	GPS/ГЛОНАСС	646
2	A002AA78	ГЛОНАСС	556
2	A002AA78	Инерциальная система	594
3	A003AA79	GPS	373
3	A003AA79	GPS/ГЛОНАСС	359
3	A003AA79	ГЛОНАСС	338
3	A003AA79	Инерциальная система	375
4	A004AA80	GPS	500
4	A004AA80	GPS/ГЛОНАСС	537
4	A004AA80	ГЛОНАСС	539
4	A004AA80	Инерциальная система	542
5	A005AA81	GPS	336
5	A005AA81	GPS/ГЛОНАСС	302
5	A005AA81	ГЛОНАСС	340
5	A005AA81	Инерциальная система	289
6	A006AA77	GPS	678

Рис. 26: Результат выполнения запроса 8

Время выполнения запроса:

Planning Time: 0.432 ms

Execution Time: 139.903 ms

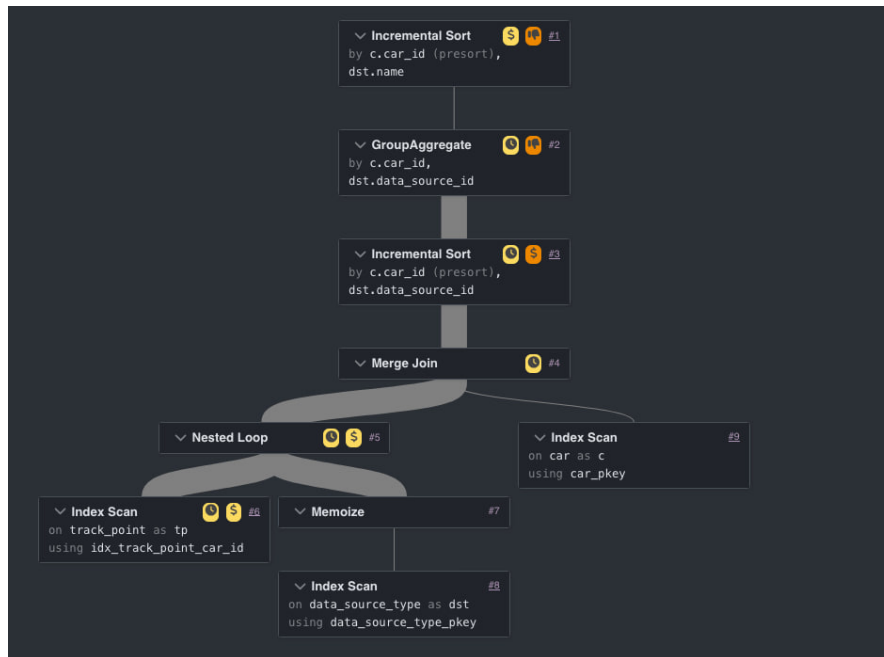


Рис. 27: Последовательность выполнения запроса 8

По индексу читаются все точки трека. Для каждой точки через кэшированный поиск (Memoize) определяется тип источника данных — это позволяет не обращаться к таблице **data_source_type** повторно для одного и того же значения. Затем точки трека соединяются с автомобилями через Merge Join. Данные инкрементально сортируются по (**car_id**, **data_source_id**) и группируются: для каждой пары считается количество координат. В конце результат сортируется по **car_id** и названию источника.

Реляционная формула запроса:

$$\pi_{\text{car_id, reg_number, dst.name, coord_count}} \left(\gamma_{\text{car_id, reg_number, data_source_id, dst.name; COUNT(track_point_id) \rightarrow coord_count}} \left(\begin{array}{l} \text{data_source_type} \\ \bowtie_{\text{data_source_id}} \text{ track_point} \\ \bowtie_{\text{car_id}} \text{ car} \end{array} \right) \right)$$

Запрос 8 — Количество координат по автомобилю и источнику данных

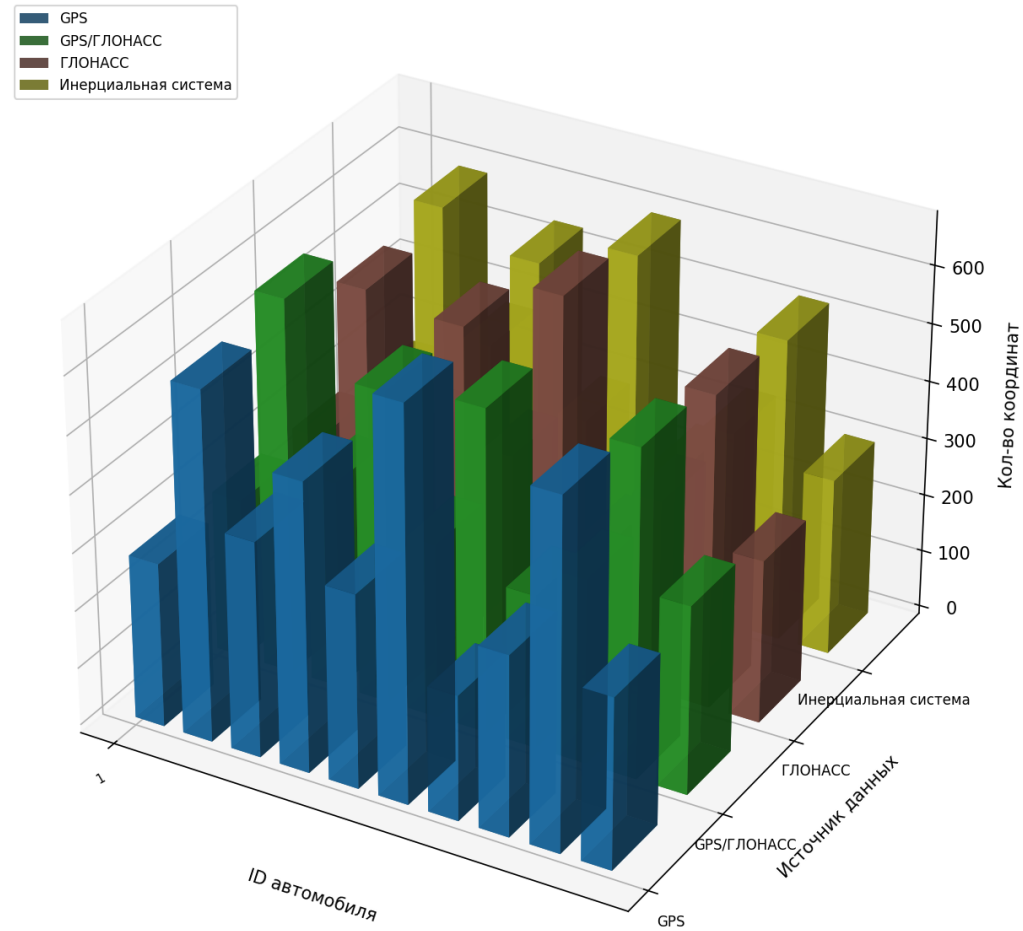


Рис. 28: 3D-гистограмма запроса 8 — количество координат по автомобилю и источнику данных

4.10. Запрос 9

Для всех треков автомобиля "А", поменять статус трека с "А" на "В".

```
1 UPDATE track
2 SET track_status_id = (
3     SELECT track_status_id FROM track_status_type WHERE name =
4     'Завершён'
5 )
6 WHERE car_id = (SELECT car_id FROM car WHERE reg_number =
7     'AAA00177')
8 AND track_status_id = (
9     SELECT track_status_id FROM track_status_type WHERE name =
10    'АКТИВНЫЙ'
11 );
```

Listing 10: Запрос 9

Время выполнения запроса:

Planning Time: 0.266 ms

Execution Time: 0.078 ms

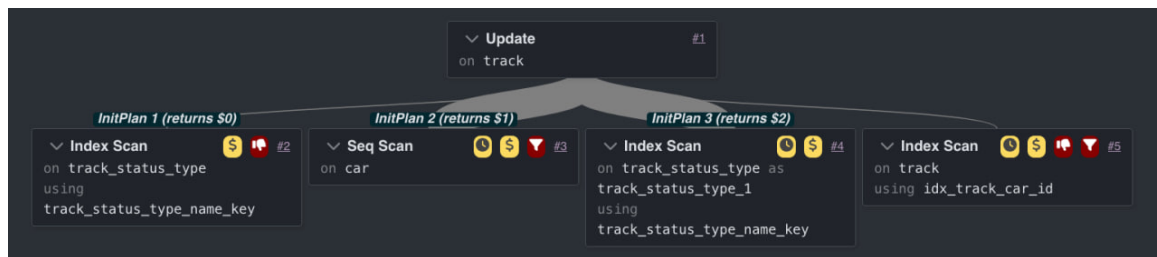


Рис. 29: Последовательность выполнения запроса 9

Три подзапроса выполняются независимо до начала основного обновления. Первый находит **track_status_id** нового статуса по индексу (не выполняется, если подходящих треков нет). Второй последовательно читает **car** и находит **car_id** по номеру автомобиля. Третий находит **track_status_id** текущего статуса по индексу. После этого по индексу находятся все треки нужного автомобиля с нужным текущим статусом, и для каждой такой строки выполняется обновление поля **track_status_id**.

Реляционная формула операции обновления.

Пусть:

$$\begin{aligned} C &= \pi_{\text{car_id}} \left(\sigma_{\text{reg_number}=A}(\text{car}) \right), \\ S_B &= \pi_{\text{track_status_id}} \left(\sigma_{\text{name}=B}(\text{track_status_type}) \right), \\ S_C &= \pi_{\text{track_status_id}} \left(\sigma_{\text{name}=C}(\text{track_status_type}) \right). \end{aligned}$$

```

UPDATE track_status_id=s_C (
    σ car_id=c ∧ track_status_id=s_B (
        track
    )
)

```

Заключение

В ходе работы выполнен анализ предметной области «Геомониторинг автопарка каршеринга», определены цели функционирования базы данных, выявлены сущности и их атрибуты. На основе анализа построены ER-диаграмма и схема базы данных, описаны таблицы с обоснованием типов атрибутов.

Схема реализована в СУБД PostgreSQL. Для заполнения базы данных написан seeder на языке Go с использованием библиотек **pgx/v5** и **golang-migrate**. База данных содержит 20 таблиц, в которой хранится 364523 записи.

Выполнены 9 запросов к базе данных. Для каждого запроса приведены результат выполнения запроса, графический план выполнения, текстовое описание и реляционная формула.

Работа выполнялась на устройстве Apple MacBook Air с процессором Apple M4 и 16 ГБ оперативной памяти. В Docker-контейнере поднимался PostgreSQL 14. Генерация данных реализована на языке Go с использованием библиотек **pgx/v5** и **golang-migrate**, визуализация результатов выполнена средствами Python (**matplotlib**).

Список литературы

- [1] Попов С.Г. Лист самостоятельной проверки хода выполнения курсовой работы по дисциплине «Теоретические основы баз данных». 4 семестр.
- [2] PostgreSQL 16 Documentation [Электронный ресурс]: The PostgreSQL Global Development Group. URL: <https://www.postgresql.org/docs/16> (Дата обращения: 09.05.2026).
- [3] Takahashi M., Azuma S., Trend C.L. The Manga Guide to Databases. — No Starch Press, 2009. — 214 p.
- [4] pgx - PostgreSQL Driver and Toolkit URL: <https://github.com/jackc/pgx> (Дата обращения: 20.04.2026).
- [5] Axenta. Мониторинг транспорта в каршеринге и корпоративном автопарке. — 2026. URL: <https://axenta.tech/blog/novosti/monitoring-transporta-v-karsheringe-i-korporativnom-avtoparke> (Дата обращения: 20.03.2026).
- [6] Navixy. Что такое телематика? Подробный гид для IoT, GPS и транспорта. — 2020. URL: <https://www.navixy.com/ru/blog/what-is-telematics/> (Дата обращения: 20.03.2026).
- [7] РБК. Совместная мобильность: чего ждать от рынка каршеринга России. — 2024. — URL: <https://www.rbc.ru/industries/news/661e8a3a9a79478be5eab43d> (Дата обращения: 20.03.2026).
- [8] Система управления базами данных [Электронный ресурс]: Википедия. — URL: https://ru.wikipedia.org/wiki/%D0%A1%D0%B8%D1%81%D1%82%D0%B5%D0%BC%D0%B0_%D1%83%D0%BF%D1%80%D0%B0%D0%B2%D0%BB%D0%B5%D0%BD%D0%B8%D1%8F_%D0%B1%D0%B0%D0%B7%D0%B0%D0%BC%D0%B8_%D0%B4%D0%B0%D0%BD%D0%BD%D1%8B%D1%85 (Дата обращения: 20.03.2026).

Приложение А «Исходный код программы генерации схемы базы данных»

```
1 CREATE TABLE employee_type (  
2     employee_type_id SERIAL PRIMARY KEY,  
3     name VARCHAR(30) NOT NULL UNIQUE,  
4     description TEXT  
5 );  
6  
7 CREATE TABLE employee_status (  
8     employee_status_id SERIAL PRIMARY KEY,  
9     name VARCHAR(20) NOT NULL UNIQUE  
10 );  
11  
12 CREATE TABLE employee (  
13     employee_id SERIAL PRIMARY KEY,  
14     employee_type_id INT NOT NULL REFERENCES  
15     employee_type(employee_type_id),  
16     full_name VARCHAR(100) NOT NULL,  
17     login VARCHAR(30) NOT NULL UNIQUE,  
18     employee_status_id INT NOT NULL REFERENCES  
19     employee_status(employee_status_id)  
20 );  
21  
22 CREATE TABLE parking_zone_type (  
23     zone_type_id SERIAL PRIMARY KEY,  
24     name VARCHAR(20) NOT NULL UNIQUE  
25 );  
26  
27 CREATE TABLE parking_zone_status (  
28     zone_status_id SERIAL PRIMARY KEY,  
29     name VARCHAR(25) NOT NULL UNIQUE  
30 );  
31  
32 CREATE TABLE parking_zone (  
33     parking_zone_id SERIAL PRIMARY KEY,  
34     name VARCHAR(60) NOT NULL,  
35     zone_type_id INT NOT NULL REFERENCES  
36     parking_zone_type(zone_type_id),  
37     city_district VARCHAR(50) NOT NULL,  
38     max_cars INT NOT NULL,  
39     zone_status_id INT NOT NULL REFERENCES  
40     parking_zone_status(zone_status_id)  
41 );  
42  
43 CREATE TABLE parking_zone_point (  
44     parking_zone_point_id SERIAL PRIMARY KEY,  
45     parking_zone_id INT NOT NULL REFERENCES  
46     parking_zone(parking_zone_id) ON DELETE CASCADE,  
47     vertex_number INT NOT NULL,  
48     latitude NUMERIC(8, 5) NOT NULL,  
49     longitude NUMERIC(8, 5) NOT NULL,
```

```

45     UNIQUE (parking_zone_id, vertex_number)
46 );
47
48 CREATE TABLE car_status (
49     car_status_id      SERIAL PRIMARY KEY,
50     name               VARCHAR(25) NOT NULL UNIQUE
51 );
52
53 CREATE TABLE car (
54     car_id             SERIAL PRIMARY KEY,
55     reg_number         VARCHAR(20) NOT NULL UNIQUE,
56     brand              VARCHAR(50) NOT NULL,
57     model              VARCHAR(70) NOT NULL,
58     manufacture_year   INT NOT NULL,
59     vin                VARCHAR(17) NOT NULL UNIQUE,
60     car_status_id      INT NOT NULL REFERENCES
61     car_status(car_status_id)
62 );
63
64 CREATE TABLE parking_session (
65     parking_session_id SERIAL PRIMARY KEY,
66     car_id             INT NOT NULL REFERENCES car(car_id) ON
67     DELETE CASCADE,
68     parking_zone_id    INT NOT NULL REFERENCES
69     parking_zone(parking_zone_id) ON DELETE CASCADE,
70     entry_time         TIMESTAMP NOT NULL,
71     exit_time          TIMESTAMP
72 );
73
74 CREATE TABLE track_status_type (
75     track_status_id    SERIAL PRIMARY KEY,
76     name               VARCHAR(20) NOT NULL UNIQUE
77 );
78
79 CREATE TABLE track_kind_type (
80     track_kind_id      SERIAL PRIMARY KEY,
81     name               VARCHAR(25) NOT NULL UNIQUE
82 );
83
84 CREATE TABLE data_source_type (
85     data_source_id     SERIAL PRIMARY KEY,
86     name               VARCHAR(25) NOT NULL UNIQUE
87 );
88
89 CREATE TABLE track (
90     track_id           SERIAL PRIMARY KEY,
91     car_id             INT NOT NULL REFERENCES car(car_id) ON
92     DELETE CASCADE,
93     start_time         TIMESTAMP NOT NULL,
94     end_time           TIMESTAMP,
95     track_status_id    INT NOT NULL REFERENCES
96     track_status_type(track_status_id),

```

```

92     track_kind_id      INT NOT NULL REFERENCES
    track_kind_type(track_kind_id)
93 );
94
95 CREATE TABLE track_point (
96     track_point_id     SERIAL PRIMARY KEY,
97     track_id           INT REFERENCES track(track_id) ON DELETE
    CASCADE,
98     car_id             INT REFERENCES car(car_id) ON DELETE
    CASCADE,
99     latitude           NUMERIC(8, 5) NOT NULL,
100    longitude          NUMERIC(8, 5) NOT NULL,
101    speed_kmh          NUMERIC(5, 2),
102    data_source_id     INT NOT NULL REFERENCES
    data_source_type(data_source_id)
103 );
104
105 CREATE TABLE alert_event_type (
106     alert_event_type_id SERIAL PRIMARY KEY,
107     name              VARCHAR(25) NOT NULL UNIQUE,
108     description       TEXT
109 );
110
111 CREATE TABLE alert_event_process_status (
112     status_id         SERIAL PRIMARY KEY,
113     name              VARCHAR(20) NOT NULL UNIQUE
114 );
115
116 CREATE TABLE alert_event (
117     alert_event_id     SERIAL PRIMARY KEY,
118     car_id            INT NOT NULL REFERENCES car(car_id) ON
    DELETE CASCADE,
119     employee_id       INT REFERENCES employee(employee_id),
120     alert_event_type_id INT NOT NULL REFERENCES
    alert_event_type(alert_event_type_id),
121     latitude          NUMERIC(8, 5),
122     longitude         NUMERIC(8, 5),
123     description       TEXT,
124     status_id         INT NOT NULL REFERENCES
    alert_event_process_status(status_id)
125 );
126
127 CREATE TABLE geo_request_type (
128     geo_request_type_id SERIAL PRIMARY KEY,
129     name              VARCHAR(25) NOT NULL UNIQUE,
130     description       TEXT
131 );
132
133 CREATE TABLE geo_request (
134     geo_request_id     SERIAL PRIMARY KEY,
135     employee_id       INT NOT NULL REFERENCES
    employee(employee_id),
136     car_id            INT NOT NULL REFERENCES car(car_id),

```



```

137     geo_request_type_id INT NOT NULL REFERENCES
        geo_request_type(geo_request_type_id),
138     request_goal          TEXT NOT NULL
139 );

```

Listing 11: SQL-скрипт для создания схемы базы данных

```

1  -- employee
2  CREATE INDEX IF NOT EXISTS idx_employee_employee_type_id
3      ON employee(employee_type_id);
4
5  CREATE INDEX IF NOT EXISTS idx_employee_employee_status_id
6      ON employee(employee_status_id);
7
8  -- parking_zone
9  CREATE INDEX IF NOT EXISTS idx_parking_zone_zone_type_id
10     ON parking_zone(zone_type_id);
11
12  CREATE INDEX IF NOT EXISTS idx_parking_zone_zone_status_id
13     ON parking_zone(zone_status_id);
14
15  -- parking_zone_point
16  CREATE INDEX IF NOT EXISTS idx_parking_zone_point_parking_zone_id
17     ON parking_zone_point(parking_zone_id);
18
19  -- car
20  CREATE INDEX IF NOT EXISTS idx_car_car_status_id
21     ON car(car_status_id);
22
23  -- parking_session
24  CREATE INDEX IF NOT EXISTS idx_parking_session_car_id
25     ON parking_session(car_id);
26
27  CREATE INDEX IF NOT EXISTS idx_parking_session_parking_zone_id
28     ON parking_session(parking_zone_id);
29
30  -- track
31  CREATE INDEX IF NOT EXISTS idx_track_car_id
32     ON track(car_id);
33
34  CREATE INDEX IF NOT EXISTS idx_track_track_status_id
35     ON track(track_status_id);
36
37  CREATE INDEX IF NOT EXISTS idx_track_track_kind_id
38     ON track(track_kind_id);
39
40  -- track_point
41  CREATE INDEX IF NOT EXISTS idx_track_point_track_id
42     ON track_point(track_id);
43
44  CREATE INDEX IF NOT EXISTS idx_track_point_car_id
45     ON track_point(car_id);
46

```

```

47 CREATE INDEX IF NOT EXISTS idx_track_point_data_source_id
48     ON track_point(data_source_id);
49
50 -- alert_event
51 CREATE INDEX IF NOT EXISTS idx_alert_event_car_id
52     ON alert_event(car_id);
53
54 CREATE INDEX IF NOT EXISTS idx_alert_event_employee_id
55     ON alert_event(employee_id);
56
57 CREATE INDEX IF NOT EXISTS idx_alert_event_alert_event_type_id
58     ON alert_event(alert_event_type_id);
59
60 CREATE INDEX IF NOT EXISTS idx_alert_event_status_id
61     ON alert_event(status_id);
62
63 -- geo_request
64 CREATE INDEX IF NOT EXISTS idx_geo_request_employee_id
65     ON geo_request(employee_id);
66
67 CREATE INDEX IF NOT EXISTS idx_geo_request_car_id
68     ON geo_request(car_id);
69
70 CREATE INDEX IF NOT EXISTS idx_geo_request_geo_request_type_id
71     ON geo_request(geo_request_type_id);
72
73 -- DROP индексов
74 DROP INDEX IF EXISTS idx_employee_employee_type_id;
75 DROP INDEX IF EXISTS idx_employee_employee_status_id;
76
77 DROP INDEX IF EXISTS idx_parking_zone_zone_type_id;
78 DROP INDEX IF EXISTS idx_parking_zone_zone_status_id;
79
80 DROP INDEX IF EXISTS idx_parking_zone_point_parking_zone_id;
81
82 DROP INDEX IF EXISTS idx_car_car_status_id;
83
84 DROP INDEX IF EXISTS idx_parking_session_car_id;
85 DROP INDEX IF EXISTS idx_parking_session_parking_zone_id;
86
87 DROP INDEX IF EXISTS idx_track_car_id;
88 DROP INDEX IF EXISTS idx_track_track_status_id;
89 DROP INDEX IF EXISTS idx_track_track_kind_id;
90
91 DROP INDEX IF EXISTS idx_track_point_track_id;
92 DROP INDEX IF EXISTS idx_track_point_car_id;
93 DROP INDEX IF EXISTS idx_track_point_data_source_id;
94
95 DROP INDEX IF EXISTS idx_alert_event_car_id;
96 DROP INDEX IF EXISTS idx_alert_event_employee_id;
97 DROP INDEX IF EXISTS idx_alert_event_alert_event_type_id;
98 DROP INDEX IF EXISTS idx_alert_event_status_id;
99

```

```
100 DROP INDEX IF EXISTS idx_geo_request_employee_id;  
101 DROP INDEX IF EXISTS idx_geo_request_car_id;  
102 DROP INDEX IF EXISTS idx_geo_request_geo_request_type_id;
```

Listing 12: SQL-скрипт для создания индексов

Приложение Б «Исходный код программы заполнения базы данных тестовыми данными»

```
1 package main
2
3 import (
4     "context"
5     "db_generation/internal/config"
6     "db_generation/internal/logger"
7     "db_generation/internal/postgres"
8     "db_generation/internal/seeder"
9     "sort"
10    "time"
11
12    "go.uber.org/zap"
13 )
14
15 func main() {
16     ctx, cancel := context.WithTimeout(context.Background(),
17         20*time.Minute)
18     defer cancel()
19
20     ctx, err := logger.New(ctx)
21     if err != nil {
22         panic("failed to create logger")
23     }
24
25     log, err := logger.GetLoggerFromCtx(ctx)
26     if err != nil {
27         panic("failed to create logger")
28     }
29
30     cfg, err := config.New()
31     if err != nil {
32         log.Fatal(ctx, "failed to parse config")
33     }
34
35     pool, err := postgres.New(ctx, cfg.Postgres)
36     if err != nil {
37         log.Fatal(ctx, "failed to create postgres pool")
38     }
39     defer pool.Close()
40
41     if err := postgres.Migrate(ctx, cfg.Postgres); err != nil {
42         log.Fatal(ctx, "failed to run migrations")
43     }
44
45     log.Info(ctx, "db migration success")
46
47     report, err := seeder.SeedWithReport(ctx, pool,
48         seeder.DefaultConfig())
49     if err != nil {
```

```

48     log.Fatal(ctx, "failed to seed database", zap.Error(err))
49 }
50
51 tables := make([]string, 0, len(report.TableCounts))
52 for table := range report.TableCounts {
53     tables = append(tables, table)
54 }
55 sort.Strings(tables)
56
57 for _, table := range tables {
58     log.Info(ctx, "seed report", zap.String("table", table),
59         zap.Int("count", report.TableCounts[table]))
60 }
61 log.Info(ctx, "seed report total", zap.Int("total_rows",
62     report.TotalRows))
63 }

```

Listing 13: Точка входа программы (cmd/seeder/main.go)

```

1 package config
2
3 import (
4     "db_generation/internal/postgres"
5     "fmt"
6     "os"
7
8     "github.com/ilyakaznacheev/cleanenv"
9 )
10
11 type Config struct {
12     Postgres postgres.Config `yaml:"postgres"`
13 }
14
15 func New() (*Config, error) {
16     var cfg Config
17
18     configPaths := []string{
19         "./configs/configs.yaml",
20         "./configs/config.yaml",
21         "configs/configs.yaml",
22         "configs/config.yaml",
23         "../configs/configs.yaml",
24         "../configs/config.yaml",
25         "../../configs/configs.yaml",
26         "../../configs/config.yaml",
27     }
28
29     var configPath string
30     for _, path := range configPaths {
31         if _, err := os.Stat(path); err == nil {
32             configPath = path
33             break
34         }
35     }
36
37     if configPath == "" {
38         return &Config{}, fmt.Errorf("config file not found")
39     }
40
41     if err := cleanenv.ReadConfig(configPath, &cfg); err != nil {
42         return &Config{}, fmt.Errorf("error reading config: %w", err)
43     }
44
45     return &cfg, nil
46 }

```

Listing 14: Загрузка конфигурации (internal/config/config.go)

```

1 package logger
2
3 import (
4     "context"
5     "fmt"
6
7     "go.uber.org/zap"
8 )
9
10 type key string
11
12 const (
13     KeyForLogger    key = "logger"
14     KeyForRequestID key = "requestID"
15 )
16
17 type Logger struct {
18     l *zap.Logger
19 }
20
21 func New(ctx context.Context) (context.Context, error) {
22     l, err := zap.NewProduction()
23     if err != nil {
24         return ctx, fmt.Errorf("failed to create logger: %w", err)
25     }
26     wrapped := &Logger{l: l}
27     ctx = context.WithValue(ctx, KeyForLogger, wrapped)
28     return ctx, nil
29 }
30
31 func GetLoggerFromCtx(ctx context.Context) (*Logger, error) {
32     if l, ok := ctx.Value(KeyForLogger).(*Logger); ok {
33         return l, nil
34     }
35     return nil, fmt.Errorf("logger not found in context")
36 }
37
38 func (l *Logger) Debug(ctx context.Context, msg string, fields
...zap.Field) {
39     if ctx.Value(KeyForRequestID) != nil {
40         fields = append(fields, zap.String(string(KeyForRequestID),
ctx.Value(KeyForRequestID).(string)))
41     }
42     l.l.Debug(msg, fields...)
43 }
44
45 func (l *Logger) Info(ctx context.Context, msg string, fields
...zap.Field) {
46     if ctx.Value(KeyForRequestID) != nil {
47         fields = append(fields, zap.String(string(KeyForRequestID),
ctx.Value(KeyForRequestID).(string)))
48     }

```

```

49     l.l.Info(msg, fields...)
50 }
51
52 func (l *Logger) Warn(ctx context.Context, msg string, fields
    ...zap.Field) {
53     if ctx.Value(KeyForRequestID) != nil {
54         fields = append(fields, zap.String(string(KeyForRequestID),
            ctx.Value(KeyForRequestID).(string)))
55     }
56     l.l.Warn(msg, fields...)
57 }
58
59 func (l *Logger) Error(ctx context.Context, msg string, fields
    ...zap.Field) {
60     if ctx.Value(KeyForRequestID) != nil {
61         fields = append(fields, zap.String(string(KeyForRequestID),
            ctx.Value(KeyForRequestID).(string)))
62     }
63     l.l.Error(msg, fields...)
64 }
65
66 func (l *Logger) Fatal(ctx context.Context, msg string, fields
    ...zap.Field) {
67     if ctx.Value(KeyForRequestID) != nil {
68         fields = append(fields, zap.String(string(KeyForRequestID),
            ctx.Value(KeyForRequestID).(string)))
69     }
70     l.l.Fatal(msg, fields...)
71 }

```

Listing 15: Обёртка логгера (internal/logger/logger.go)


```

1 package postgres
2
3 import (
4     "context"
5     "db_generation/internal/logger"
6     "errors"
7     "fmt"
8     "os"
9     "path/filepath"
10    "time"
11
12    "github.com/golang-migrate/migrate/v4"
13    _ "github.com/golang-migrate/migrate/v4/database/postgres"
14    _ "github.com/golang-migrate/migrate/v4/source/file"
15    "github.com/jackc/pgx/v5/pgxpool"
16 )
17
18 type Config struct {
19     Host      string `yaml:"host"`
20     Port      string `yaml:"port"`
21     Username  string `yaml:"username"`
22     Password  string `yaml:"password"`
23     Database  string `yaml:"database"`
24     MaxConns  int32  `yaml:"max_conns" env:"MAX_CONNS"
25             env-default:"10"`
26     MinConns  int32  `yaml:"min_conns" env:"MIN_CONNS"
27             env-default:"5"`
28 }
29
30 func New(ctx context.Context, cfg Config) (*pgxpool.Pool, error) {
31     poolCfg, err := pgxpool.ParseConfig(cfg.GetConnString())
32     if err != nil {
33         return nil, fmt.Errorf("failed to parse postgres config: %w",
34             err)
35     }
36
37     poolCfg.MaxConns = cfg.MaxConns
38     poolCfg.MinConns = cfg.MinConns
39     poolCfg.MaxConnLifetime = 30 * time.Minute
40     poolCfg.MaxConnIdleTime = 5 * time.Minute
41     poolCfg.HealthCheckPeriod = 1 * time.Minute
42
43     pool, err := pgxpool.NewWithConfig(ctx, poolCfg)
44     if err != nil {
45         return nil, fmt.Errorf("failed to connect to postgres: %w",
46             err)
47     }
48
49     return pool, nil
50 }
51
52 func Migrate(ctx context.Context, cfg Config) error {

```

```

49 connString := cfg.GetConnString()
50 log, err := logger.GetLoggerFromCtx(ctx)
51 if err != nil {
52     return fmt.Errorf("failed to get logger from context: %w", err)
53 }
54
55 migrationPaths := []string{
56     "migrations",
57     "./migrations",
58     "../migrations",
59     "../../migrations",
60 }
61
62 var migrationPath string
63 for _, path := range migrationPaths {
64     if info, err := os.Stat(path); err == nil && info.IsDir() {
65         migrationPath, _ = filepath.Abs(path)
66         break
67     }
68 }
69
70 if migrationPath == "" {
71     return fmt.Errorf("migrations directory not found")
72 }
73
74 m, err := migrate.New("file://"+migrationPath, connString)
75 if err != nil {
76     return fmt.Errorf("failed to create migration instance: %w",
77         err)
78 }
79
80 err = m.Up()
81 if err != nil && !errors.Is(err, migrate.ErrNoChange) {
82     return fmt.Errorf("failed to run migrations: %w", err)
83 }
84
85 log.Info(ctx, "migrations applied successfully")
86 return nil
87 }
88
89 func (c *Config) GetConnString() string {
90     connString :=
91         fmt.Sprintf("postgres://%s:%s@%s:%s/%s?sslmode=disable",
92             c.Username,
93             c.Password,
94             c.Host,
95             c.Port,
96             c.Database,
97         )
98     return connString
99 }

```

Listing 16: Подключение к PostgreSQL и миграции (internal/postgres/postgres.go)

```

1 package seeder
2
3 import "time"
4
5 type Config struct {
6     Employees      int
7     Cars            int
8     ParkingZones    int
9     ZonePointsMin   int
10    ZonePointsMax    int
11    TracksPerCarMin  int
12    TracksPerCarMax  int
13    SessionsPerCarMin int
14    SessionsPerCarMax int
15    TrackPointsMin   int
16    TrackPointsMax   int
17    AlertEvents       int
18    AlertEventPerCarMin int
19    AlertEventPerCarMax int
20    GeoRequestsMin    int
21    GeoRequestsMax    int
22    BatchSize         int
23 }
24
25 type Report struct {
26     TableCounts map[string]int
27     TotalRows    int
28 }
29
30 type trackRef struct {
31     ID          int
32     CarID       int
33     ZoneID      int
34     StartTime   time.Time
35     EndTime     *time.Time
36 }
37
38 type level1Refs struct {
39     EmpTypeIDs      []int
40     EmpStatusIDs    []int
41     CarStatusIDs    []int
42     ZoneTypeIDs     []int
43     ZoneStatusIDs   []int
44     TrackStatusIDs  []int
45     TrackKindIDs    []int
46     DataSourceIDs   []int
47     AlertEventIDs   []int
48     AlertProcStatusIDs []int
49     GeoReqTypeIDs   []int
50 }
51
52 type level2Refs struct {

```

```
53 EmployeeIDs    []int
54 CarIDs         []int
55 ParkingZoneIDs []int
56 }
```

Listing 17: Типы данных сидера (internal/seeder/types.go)

```

1 package seeder
2
3 var empTypeData = []struct{ name, desc string }{
4     {"Оператор коллцентра-", "Обрабатывает входящие обращения
5     клиентов"},
6     {"Администратор системы", "Управляет системой и пользователями"},
7     {"Сервисный специалист", "Обслуживает автомобили и
8     оборудование"},
9     {"Аналитик", "Анализирует данные и формирует отчёты"},
10 }
11
12 var empStatusNames = []string{"Активен", "В отпуске", "Уволен",
13     "Заблокирован"}
14 var carStatusNames = []string{"Свободен", "В аренде", "На
15     обслуживании", "Неисправен"}
16 var zoneTypeNames = []string{"Бесплатная", "Платная",
17     "Запрещённая"}
18 var zoneStatusNames = []string{"Активна", "Временно закрыта",
19     "Недоступна"}
20 var trackStatusNames = []string{"Активный", "Завершён", "Архивный"}
21 var trackKindNames = []string{"Пользовательский", "Сервисный"}
22 var dataSourceNames = []string{"GPS", "ГЛОНАСС", "GPSГЛОНАСС/",
23     "Инерциальная система"}
24
25 var alertEventData = []struct{ name, desc string }{
26     {"Потеря связи", "Автомобиль не выходит на связь"},
27     {"Превышение скорости", "Скорость выше допустимого предела"},
28     {"Резкое торможение", "Зафиксировано экстренное торможение"},
29     {"Удар", "Зафиксировано столкновение"},
30     {"Попытка угона", "Несанкционированное использование"},
31     {"Выезд за границу", "Автомобиль покинул разрешённую зону"},
32 }
33
34 var alertProcStatusNames = []string{"Новое", "В работе", "Закрыто"}
35
36 var geoReqTypeData = []struct{ name, desc string }{
37     {"Текущие координаты", "Запрос текущего местоположения"},
38     {"История событий", "Запрос истории перемещений"},
39 }
40
41 var surnames = []string{
42     "Иванов", "Смирнов", "Кузнецов", "Попов", "Васильев",
43     "Петров", "Соколов", "Михайлов", "Новиков", "Фёдоров",
44     "Морозов", "Волков", "Алексеев", "Лебедев", "Семёнов",
45     "Егоров", "Павлов", "Козлов", "Степанов", "Николаев",
46     "Орлов", "Андреев", "Макаров", "Никитин", "Захаров",
47     "Зайцев", "Соловьёв", "Борисов", "Яковлев", "Григорьев",
48 }
49
50 var firstNamesMale = []string{
51     "Александр", "Алексей", "Андрей", "Артём", "Василий",
52     "Виктор", "Владимир", "Григорий", "Дмитрий", "Евгений",

```

```

46  "Иван", "Игорь", "Кирилл", "Константин", "Максим",
47  "Михаил", "Николай", "Павел", "Роман", "Сергей",
48  }
49
50  var patronymicsMale = []string{
51      "Александрович", "Алексеевич", "Андреевич", "Артёмович",
52      "Васильевич",
53      "Викторович", "Владимирович", "Григорьевич", "Дмитриевич",
54      "Евгеньевич",
55      "Иванович", "Игоревич", "Кириллович", "Константинович",
56      "Максимович",
57      "Михайлович", "Николаевич", "Павлович", "Романович", "Сергеевич",
58  }
59
60  var carData = []struct {
61      brand  string
62      models []string
63  }{
64      {"Lada", []string{"Vesta", "Granta", "Niva Travel", "Largus",
65          "Vesta SW"}},
66      {"Kia", []string{"Rio", "Sportage", "Ceed", "K5", "Sorento"}},
67      {"Hyundai", []string{"Solaris", "Tucson", "Creta", "Santa Fe",
68          "Elantra"}},
69      {"Toyota", []string{"Camry", "RAV4", "Land Cruiser", "Corolla",
70          "Highlander"}},
71      {"Volkswagen", []string{"Polo", "Tiguan", "Passat", "Taos",
72          "Touareg"}},
73      {"Renault", []string{"Logan", "Sandero", "Duster", "Arkana",
74          "Kaptur"}},
75      {"Nissan", []string{"Qashqai", "X-Trail", "Juke", "Almera",
76          "Pathfinder"}},
77      {"Skoda", []string{"Octavia", "Rapid", "Superb", "Kodiaq",
78          "Karoq"}},
79      {"Mazda", []string{"3", "6", "CX-5", "CX-9", "MX-5"}},
80      {"BMW", []string{"3 Series", "5 Series", "X5", "X3", "7
81          Series"}},
82  }
83
84  var plateLetters = []rune("АВЕКМНОРСТУХ")
85
86  var zoneNamePrefixes = []string{
87      "ТЦ Мера", "ТЦ Галерея", "ТЦ Авиапарк", "ТЦ Columbus",
88      "ТЦ Европейский", "ТЦ Атриум", "ТЦ Рио", "ТЦ Ереван Плаза",
89      "БЦ МоскваСити-", "Парк Горького", "ВДНХ", "Красная площадь",
90      "Лужники", "Сокольники", "Измайловский парк", "Коломенское",
91      "ЖК Северный", "ЖК Западный", "МЦД Одинцово", "МЦД Подольск",
92  }
93
94  var zoneNameSuffixes = []string{
95      "северный въезд", "южный въезд", "восточный въезд", "западный
96      въезд",
97      "парковка А", "парковка Б", "парковка В", "подземная парковка",
98      "уровень 1", "уровень 2", "главный вход", "боковой вход",

```

```
87 }  
88  
89 var moscowDistricts = []string{  
90     "Центральный", "Северный", "СевероВосточный-", "Восточный",  
91     "ЮгоВосточный-", "Южный", "ЮгоЗападный-", "Западный",  
92     "СевероЗападный-",  
93 }
```

Listing 18: Статические данные для генерации (internal/seeder/data.go)

```

1 package seeder
2
3 import (
4     "fmt"
5     "strings"
6 )
7
8 func genRegNumber(i int) string {
9     chars := []rune("ABEKMHOPTVX")
10    n := i % 999
11    c1 := chars[(i/999)%len(chars)]
12    c2 := chars[(i/(999*len(chars))%len(chars)]
13    c3 := chars[(i/(999*len(chars)*len(chars))%len(chars)]
14    region := 77 + (i % 5)
15    return fmt.Sprintf("%c%03d%c%c%d", c1, n+1, c2, c3, region)
16 }
17
18 func transliterate(s string) string {
19     mapping := map[rune]string{
20         A'': "A", Б'': "B", В'': "V", Г'': "G", Д'': "D", Е'': "E",
21         Ё'': "E", Ж'': "ZH",
22         З'': "Z", И'': "I", Й'': "Y", К'': "K", Л'': "L", М'': "M",
23         Н'': "N", О'': "O",
24         П'': "P", Р'': "R", С'': "S", Т'': "T", У'': "U", Ф'': "F",
25         Х'': "H", Ц'': "TS",
26         Ч'': "CH", Ш'': "SH", Щ'': "SCH", Ъ'': "", Ы'': "Y", Ь'': "",
27         Э'': "E", Ю'': "YU", Я'': "YA",
28         а'': "a", б'': "b", в'': "v", г'': "g", д'': "d", е'': "e",
29         ё'': "e", ж'': "zh",
30         з'': "z", и'': "i", й'': "y", к'': "k", л'': "l", м'': "m",
31         н'': "n", о'': "o",
32         п'': "p", р'': "r", с'': "s", т'': "t", у'': "u", ф'': "f",
33         х'': "h", ц'': "ts",
34         ч'': "ch", ш'': "sh", щ'': "sch", ъ'': "", ы'': "y", ь'': "",
35         э'': "e", ю'': "yu", я'': "ya",
36     }
37     var sb strings.Builder
38     for _, r := range s {
39         if val, ok := mapping[r]; ok {
40             sb.WriteString(val)
41         } else {
42             sb.WriteRune(r)
43         }
44     }
45     return sb.String()
46 }

```

Listing 19: Вспомогательные функции (internal/seeder/utlis.go)


```

1 package seeder
2
3 import (
4     "context"
5     "fmt"
6
7     "github.com/jackc/pgx/v5/pgxpool"
8 )
9
10 func DefaultConfig() Config {
11     return Config{
12         Employees:      30,
13         Cars:            200,
14         ParkingZones:    200,
15         ZonePointsMin:   20,
16         ZonePointsMax:   30,
17         TracksPerCarMin: 10,
18         TracksPerCarMax: 30,
19         SessionsPerCarMin: 10,
20         SessionsPerCarMax: 20,
21         TrackPointsMin:   70,
22         TrackPointsMax:   100,
23         AlertEvents:      500,
24         AlertEventPerCarMin: 1,
25         AlertEventPerCarMax: 3,
26         GeoRequestsMin:    8,
27         GeoRequestsMax:    20,
28         BatchSize:         1000,
29     }
30 }
31
32 var reportTables = []string{
33     "employee_type", "employee_status",
34     "car_status", "parking_zone_type", "parking_zone_status",
35     "track_status_type", "track_kind_type", "data_source_type",
36     "alert_event_type", "alert_event_process_status",
37     "geo_request_type",
38     "employee", "car", "parking_zone",
39     "parking_zone_point", "track", "parking_session",
40     "alert_event", "geo_request", "track_point",
41 }
42
43 func buildReport(ctx context.Context, pool *pgxpool.Pool)
44     (*Report, error) {
45     report := &Report{TableCounts: make(map[string]int,
46         len(reportTables))}
47     for _, table := range reportTables {
48         var count int
49         if err := pool.QueryRow(ctx, fmt.Sprintf("SELECT COUNT(*)
50             FROM %s", table)).Scan(&count); err != nil {
51             return nil, fmt.Errorf("count %s: %w", table, err)
52         }
53     }
54 }

```

```
49     report.TableCounts[table] = count
50     report.TotalRows += count
51 }
52 return report, nil
53 }
```

Listing 20: Конфигурация и отчёт сидера (internal/seeder/config.go)

```

1 package seeder
2
3 import (
4     "context"
5     "fmt"
6     "math/rand"
7     "time"
8
9     "github.com/jackc/pgx/v5/pgxpool"
10 )
11
12 func Seed(ctx context.Context, pool *pgxpool.Pool) error {
13     _, err := SeedWithReport(ctx, pool, DefaultConfig())
14     return err
15 }
16
17 func SeedWithReport(ctx context.Context, pool *pgxpool.Pool, cfg
18     Config) (*Report, error) {
19     r := rand.New(rand.NewSource(time.Now().UnixNano()))
20
21     if err := truncateAll(ctx, pool); err != nil {
22         return nil, err
23     }
24
25     fmt.Println("=== Уровень 1: справочники ===")
26     l1, n1, err := seedLevel1(ctx, pool, r)
27     if err != nil {
28         return nil, fmt.Errorf("level1: %w", err)
29     }
30     fmt.Printf(" Вставлено: %d\n\n", n1)
31
32     fmt.Println("=== Уровень 2: основные сущности ===")
33     l2, n2, err := seedLevel2(ctx, pool, r, cfg, l1)
34     if err != nil {
35         return nil, fmt.Errorf("level2: %w", err)
36     }
37     fmt.Printf(" Вставлено: %d\n\n", n2)
38
39     fmt.Println("=== Уровень 3: зависимые сущности ===")
40     tracks, n3, err := seedLevel3(ctx, pool, r, cfg, l1, l2)
41     if err != nil {
42         return nil, fmt.Errorf("level3: %w", err)
43     }
44     fmt.Printf(" Вставлено: %d\n\n", n3)
45
46     fmt.Println("=== Уровень 4: точки треков ===")
47     n4, err := seedLevel4(ctx, pool, r, cfg, tracks,
48         l1.DataSourceIDs)
49     if err != nil {
50         return nil, fmt.Errorf("level4: %w", err)
51     }
52     fmt.Printf(" Вставлено: %d\n\n", n4)

```

```

51
52     fmt.Printf("=== Итого вставлено: %d ===\n", n1+n2+n3+n4)
53     return buildReport(ctx, pool)
54 }
55
56 func truncateAll(ctx context.Context, pool *pgxpool.Pool) error {
57     query := `TRUNCATE TABLE
58         geo_request,
59         alert_event,
60         track_point,
61         parking_session,
62         track,
63         car,
64         parking_zone_point,
65         parking_zone,
66         employee,
67         employee_type,
68         employee_status,
69         car_status,
70         parking_zone_type,
71         parking_zone_status,
72         track_status_type,
73         track_kind_type,
74         data_source_type,
75         alert_event_type,
76         alert_event_process_status,
77         geo_request_type
78     RESTART IDENTITY CASCADE`
79
80     if _, err := pool.Exec(ctx, query); err != nil {
81         return fmt.Errorf("truncate tables: %w", err)
82     }
83     return nil
84 }

```

Listing 21: Главная функция заполнения БД (internal/seeder/seeder.go)

```

1 package seeder
2
3 import (
4     "context"
5     "fmt"
6     "math/rand"
7
8     "github.com/jackc/pgx/v5"
9     "github.com/jackc/pgx/v5/pgxpool"
10 )
11
12 func seedLevel1(ctx context.Context, pool *pgxpool.Pool, _
13     *rand.Rand) (*level1Refs, int, error) {
14     tx, err := pool.Begin(ctx)
15     if err != nil {
16         return nil, 0, fmt.Errorf("level1 begin: %w", err)
17     }
18     defer tx.Rollback(ctx)
19
20     refs := &level1Refs{}
21     n := 0
22
23     refs.EmpTypeID, err = insertLookupWithDesc(ctx, tx,
24         "employee_type", "employee_type_id", empTypeData)
25     if err != nil {
26         return nil, 0, err
27     }
28     n += len(refs.EmpTypeID)
29
30     refs.EmpStatusID, err = insertLookupNames(ctx, tx,
31         "employee_status", "employee_status_id", empStatusNames)
32     if err != nil {
33         return nil, 0, err
34     }
35     n += len(refs.EmpStatusID)
36
37     refs.CarStatusID, err = insertLookupNames(ctx, tx,
38         "car_status", "car_status_id", carStatusNames)
39     if err != nil {
40         return nil, 0, err
41     }
42     n += len(refs.CarStatusID)
43
44     refs.ZoneTypeID, err = insertLookupNames(ctx, tx,
45         "parking_zone_type", "zone_type_id", zoneTypeNames)
46     if err != nil {
47         return nil, 0, err
48     }
49     n += len(refs.ZoneTypeID)
50
51     refs.ZoneStatusID, err = insertLookupNames(ctx, tx,
52         "parking_zone_status", "zone_status_id", zoneStatusNames)

```

```

47  if err != nil {
48      return nil, 0, err
49  }
50  n += len(refs.ZoneStatusIDs)
51
52  refs.TrackStatusIDs, err = insertLookupNames(ctx, tx,
53      "track_status_type", "track_status_id", trackStatusNames)
54  if err != nil {
55      return nil, 0, err
56  }
57  n += len(refs.TrackStatusIDs)
58
59  refs.TrackKindIDs, err = insertLookupNames(ctx, tx,
60      "track_kind_type", "track_kind_id", trackKindNames)
61  if err != nil {
62      return nil, 0, err
63  }
64  n += len(refs.TrackKindIDs)
65
66  refs.DataSourceIDs, err = insertLookupNames(ctx, tx,
67      "data_source_type", "data_source_id", dataSourceNames)
68  if err != nil {
69      return nil, 0, err
70  }
71  n += len(refs.DataSourceIDs)
72
73  refs.AlertEventTypeIDs, err = insertLookupWithDesc(ctx, tx,
74      "alert_event_type", "alert_event_type_id", alertEventData)
75  if err != nil {
76      return nil, 0, err
77  }
78  n += len(refs.AlertEventTypeIDs)
79
80  refs.AlertProcStatusIDs, err = insertLookupNames(ctx, tx,
81      "alert_event_process_status", "status_id", alertProcStatusNames)
82  if err != nil {
83      return nil, 0, err
84  }
85  n += len(refs.AlertProcStatusIDs)
86
87  refs.GeoReqTypeIDIDs, err = insertLookupWithDesc(ctx, tx,
88      "geo_request_type", "geo_request_type_id", geoReqTypeData)
89  if err != nil {
90      return nil, 0, err
91  }
92  n += len(refs.GeoReqTypeIDIDs)
93
94  if err = tx.Commit(ctx); err != nil {
95      return nil, 0, fmt.Errorf("level1 commit: %w", err)
96  }
97  return refs, n, nil
98 }
99

```

```

94 func insertLookupWithDesc(ctx context.Context, tx pgx.Tx, table,
    idCol string, rows []struct{ name, desc string }) ([]int,
    error) {
95     ids := make([]int, 0, len(rows))
96     for _, row := range rows {
97         var id int
98         query := fmt.Sprintf(
99             "INSERT INTO %s(name, description) VALUES ($1, $2) RETURNING
    %s",
100             table, idCol,
101         )
102         if err := tx.QueryRow(ctx, query, row.name,
    row.desc).Scan(&id); err != nil {
103             return nil, fmt.Errorf("insert %s(%q): %w", table, row.name,
    err)
104         }
105         ids = append(ids, id)
106     }
107     return ids, nil
108 }
109
110 func insertLookupNames(ctx context.Context, tx pgx.Tx, table,
    idCol string, names []string) ([]int, error) {
111     ids := make([]int, 0, len(names))
112     for _, name := range names {
113         var id int
114         query := fmt.Sprintf(
115             "INSERT INTO %s(name) VALUES ($1) RETURNING %s",
116             table, idCol,
117         )
118         if err := tx.QueryRow(ctx, query, name).Scan(&id); err != nil {
119             return nil, fmt.Errorf("insert %s(%q): %w", table, name, err)
120         }
121         ids = append(ids, id)
122     }
123     return ids, nil
124 }

```

Listing 22: Заполнение таблиц-справочников (internal/seeder/level1.go)

```

1 package seeder
2
3 import (
4     "context"
5     "fmt"
6     "math/rand"
7     "time"
8
9     "github.com/jackc/pgx/v5"
10    "github.com/jackc/pgx/v5/pgxpool"
11 )
12
13 func seedLevel2(ctx context.Context, pool *pgxpool.Pool, r
14     *rand.Rand, cfg Config, ll *level1Refs) (*level2Refs, int,
15     error) {
16     tx, err := pool.Begin(ctx)
17     if err != nil {
18         return nil, 0, fmt.Errorf("level2 begin: %w", err)
19     }
20     defer tx.Rollback(ctx)
21
22     refs := &level2Refs{}
23     n := 0
24
25     refs.EmployeeIDs, err = seedEmployees(ctx, tx, r, cfg, ll)
26     if err != nil {
27         return nil, 0, err
28     }
29     n += len(refs.EmployeeIDs)
30     fmt.Printf("    employees: %d\n", len(refs.EmployeeIDs))
31
32     refs.CarIDs, err = seedCars(ctx, tx, r, cfg, ll)
33     if err != nil {
34         return nil, 0, err
35     }
36     n += len(refs.CarIDs)
37     fmt.Printf("    cars: %d\n", len(refs.CarIDs))
38
39     refs.ParkingZoneIDs, err = seedParkingZones(ctx, tx, r, cfg, ll)
40     if err != nil {
41         return nil, 0, err
42     }
43     n += len(refs.ParkingZoneIDs)
44     fmt.Printf("    parking_zones: %d\n", len(refs.ParkingZoneIDs))
45
46     if err = tx.Commit(ctx); err != nil {
47         return nil, 0, fmt.Errorf("level2 commit: %w", err)
48     }
49     return refs, n, nil
50 }
51
52 func seedEmployees(ctx context.Context, tx pgx.Tx, r *rand.Rand,

```



```

51     cfg Config, ll *level1Refs) ([]int, error) {
52     ids := make([]int, 0, cfg.Employees)
53     for i := 0; i < cfg.Employees; i++ {
54         surname := surnames[i%len(surnames)]
55         firstName := firstNamesMale[r.Intn(len(firstNamesMale))]
56         patronymic := patronymicsMale[r.Intn(len(patronymicsMale))]
57         fullName := fmt.Sprintf("%s %s %s", surname, firstName,
58         patronymic)
59         login := fmt.Sprintf("%s%02d", transliterate(surname), i+1)
60
61         var id int
62         query := "INSERT INTO employee(employee_type_id, full_name,
63         login, employee_status_id) VALUES ($1, $2, $3, $4) RETURNING
64         employee_id"
65         err := tx.QueryRow(ctx, query,
66         ll.EmpTypeID[r.Intn(len(ll.EmpTypeIDs))],
67         fullName,
68         login,
69         ll.EmpStatusID[r.Intn(len(ll.EmpStatusIDs))],
70         ).Scan(&id)
71         if err != nil {
72             return nil, fmt.Errorf("insert employee[%d]: %w", i, err)
73         }
74         ids = append(ids, id)
75     }
76     return ids, nil
77 }
78
79 func seedCars(ctx context.Context, tx pgx.Tx, r *rand.Rand, cfg
80 Config, ll *level1Refs) ([]int, error) {
81     ids := make([]int, 0, cfg.Cars)
82     currentYear := time.Now().Year()
83     minYear := currentYear - 15
84     for i := 0; i < cfg.Cars; i++ {
85         brand := carData[r.Intn(len(carData))]
86         model := brand.models[r.Intn(len(brand.models))]
87         regNum := genRegNumber(i)
88         vin := fmt.Sprintf("X%016d", i+1)
89         year := minYear + r.Intn(currentYear-minYear+1)
90
91         var id int
92         query := "INSERT INTO car(reg_number, brand, model,
93         manufacture_year, vin, car_status_id) VALUES ($1, $2, $3, $4,
94         $5, $6) RETURNING car_id"
95         err := tx.QueryRow(ctx, query,
96         regNum,
97         brand.brand,
98         model,
99         year,
100         vin,
101         ll.CarStatusID[r.Intn(len(ll.CarStatusIDs))],
102         ).Scan(&id)
103         if err != nil {

```

```

97     return nil, fmt.Errorf("insert car[%d]: %w", i, err)
98 }
99 ids = append(ids, id)
100 }
101 return ids, nil
102 }
103
104 func seedParkingZones(ctx context.Context, tx pgx.Tx, r
    *rand.Rand, cfg Config, ll *level1Refs) ([]int, error) {
105     ids := make([]int, 0, cfg.ParkingZones)
106     for i := 0; i < cfg.ParkingZones; i++ {
107         prefix := zoneNamePrefixes[r.Intn(len(zoneNamePrefixes))]
108         suffix := zoneNameSuffixes[r.Intn(len(zoneNameSuffixes))]
109         name := fmt.Sprintf("%s -- %s", prefix, suffix)
110         if len([]rune(name)) > 60 {
111             name = string([]rune(name)[:60])
112         }
113         district := moscowDistricts[r.Intn(len(moscowDistricts))]
114
115         var id int
116         query := "INSERT INTO parking_zone(name, zone_type_id,
city_district, max_cars, zone_status_id) VALUES ($1, $2, $3,
$4, $5) RETURNING parking_zone_id"
117         err := tx.QueryRow(ctx, query,
118             name,
119             ll.ZoneTypeIDs[r.Intn(len(ll.ZoneTypeIDs))],
120             district,
121             20+r.Intn(80),
122             ll.ZoneStatusIDs[r.Intn(len(ll.ZoneStatusIDs))],
123         ).Scan(&id)
124         if err != nil {
125             return nil, fmt.Errorf("insert parking_zone[%d]: %w", i, err)
126         }
127         ids = append(ids, id)
128     }
129     return ids, nil
130 }

```

Listing 23: Заполнение основных сущностей (internal/seeder/level2.go)

```

1 package seeder
2
3 import (
4     "context"
5     "fmt"
6     "math/rand"
7     "time"
8
9     "github.com/jackc/pgx/v5"
10    "github.com/jackc/pgx/v5/pgxpool"
11 )
12
13 func seedLevel3(ctx context.Context, pool *pgxpool.Pool, r
    *rand.Rand, cfg Config, l1 *level1Refs, l2 *level2Refs)
    ([]trackRef, int, error) {
14     tx, err := pool.Begin(ctx)
15     if err != nil {
16         return nil, 0, fmt.Errorf("level3 begin: %w", err)
17     }
18     defer tx.Rollback(ctx)
19
20     n := 0
21
22     pts, err := seedParkingZonePoints(ctx, tx, r, cfg,
        l2.ParkingZoneIDs)
23     if err != nil {
24         return nil, 0, err
25     }
26     n += pts
27     fmt.Printf("    parking_zone_points: %d\n", pts)
28
29     tracks, err := seedTracks(ctx, tx, r, cfg, l1, l2)
30     if err != nil {
31         return nil, 0, err
32     }
33     n += len(tracks)
34     fmt.Printf("    tracks: %d\n", len(tracks))
35
36     sessions, err := seedParkingSessions(ctx, tx, r, cfg, l2)
37     if err != nil {
38         return nil, 0, err
39     }
40     n += sessions
41     fmt.Printf("    parking_sessions: %d\n", sessions)
42
43     alerts, err := seedAlertEvents(ctx, tx, r, cfg, l1, l2)
44     if err != nil {
45         return nil, 0, err
46     }
47     n += alerts
48     fmt.Printf("    alert_events: %d\n", alerts)
49

```

```

50 geoReqs, err := seedGeoRequests(ctx, tx, r, cfg, l1, l2)
51 if err != nil {
52     return nil, 0, err
53 }
54 n += geoReqs
55 fmt.Printf("    geo_requests: %d\n", geoReqs)
56
57 if err = tx.Commit(ctx); err != nil {
58     return nil, 0, fmt.Errorf("level3 commit: %w", err)
59 }
60 return tracks, n, nil
61 }
62
63 func seedParkingZonePoints(ctx context.Context, tx pgx.Tx, r
    *rand.Rand, cfg Config, zoneIDs []int) (int, error) {
64     total := 0
65     for _, zoneID := range zoneIDs {
66         count := cfg.ZonePointsMin +
67             r.Intn(cfg.ZonePointsMax-cfg.ZonePointsMin+1)
68         for v := 1; v <= count; v++ {
69             lat := 55.7 + r.Float64()*0.2
70             lon := 37.3 + r.Float64()*0.6
71             query := "INSERT INTO parking_zone_point(parking_zone_id,
72                 vertex_number, latitude, longitude) VALUES ($1, $2, $3, $4)"
73             if _, err := tx.Exec(ctx, query, zoneID, v, lat, lon); err
74                 != nil {
75                 return 0, fmt.Errorf("insert parking_zone_point(zone=%d,
76                     v=%d): %w", zoneID, v, err)
77             }
78             total++
79         }
80     }
81     return total, nil
82 }
83
84 func seedTracks(ctx context.Context, tx pgx.Tx, r *rand.Rand, cfg
85     Config, l1 *level1Refs, l2 *level2Refs) ([]trackRef, error) {
86     refs := make([]trackRef, 0, len(l2.CarIDs)*cfg.TracksPerCarMax)
87     now := time.Now()
88
89     for _, carID := range l2.CarIDs {
90         trackCount := cfg.TracksPerCarMin +
91             r.Intn(cfg.TracksPerCarMax-cfg.TracksPerCarMin+1)
92         for j := 0; j < trackCount; j++ {
93             startTime := now.Add(-time.Duration(r.Intn(365*24)) *
94                 time.Hour)
95             duration := time.Duration(10+r.Intn(50)) * time.Minute
96             endTime := startTime.Add(duration)
97
98             statusID := l1.TrackStatusIDs[r.Intn(len(l1.TrackStatusIDs))]
99             var endPtr *time.Time
100             if statusID != l1.TrackStatusIDs[0] {
101                 endPtr = &endTime
102             }
103         }
104     }
105     return refs, nil
106 }

```

```

95     }
96
97     var id int
98     query := "INSERT INTO track(car_id, start_time, end_time,
99     track_status_id, track_kind_id) VALUES ($1, $2, $3, $4, $5)
100     RETURNING track_id"
101     err := tx.QueryRow(ctx, query,
102     carID, startTime, endPtr,
103     statusID,
104     l1.TrackKindIDs[r.Intn(len(l1.TrackKindIDs))],
105     ).Scan(&id)
106     if err != nil {
107         return nil, fmt.Errorf("insert track(car=%d, j=%d): %w",
108         carID, j, err)
109     }
110     refs = append(refs, trackRef{
111         ID:      id,
112         CarID:    carID,
113         ZoneID:
114         l2.ParkingZoneIDs[r.Intn(len(l2.ParkingZoneIDs))],
115         StartTime: startTime,
116         EndTime:    endPtr,
117     })
118 }
119 }
120 return refs, nil
121 }
122
123 func seedParkingSessions(ctx context.Context, tx pgx.Tx, r
124 *rand.Rand, cfg Config, l2 *level2Refs) (int, error) {
125     total := 0
126     now := time.Now()
127
128     for _, carID := range l2.CarIDs {
129         sessionCount := cfg.SessionsPerCarMin +
130         r.Intn(cfg.SessionsPerCarMax-cfg.SessionsPerCarMin+1)
131         for i := 0; i < sessionCount; i++ {
132             entryTime := now.Add(-time.Duration(r.Intn(365*24)) *
133             time.Hour)
134             exitTime := entryTime.Add(time.Duration(30+r.Intn(480)) *
135             time.Minute)
136             zoneID := l2.ParkingZoneIDs[r.Intn(len(l2.ParkingZoneIDs))]
137
138             query := "INSERT INTO parking_session(car_id,
139             parking_zone_id, entry_time, exit_time) VALUES ($1, $2, $3, $4)"
140             if _, err := tx.Exec(ctx, query, carID, zoneID, entryTime,
141             exitTime); err != nil {
142                 return 0, fmt.Errorf("insert parking_session(car=%d,
143                 idx=%d): %w", carID, i, err)
144             }
145             total++
146         }
147     }
148 }

```

```

137     return total, nil
138 }
139
140 func seedAlertEvents(ctx context.Context, tx pgx.Tx, r
    *rand.Rand, cfg Config, l1 *level1Refs, l2 *level2Refs) (int,
    error) {
141     inserted := 0
142     query := "INSERT INTO alert_event(car_id, employee_id,
        alert_event_type_id, latitude, longitude, description,
        status_id) VALUES ($1, $2, $3, $4, $5, $6, $7)"
143
144     if cfg.AlertEventPerCarMin > 0 || cfg.AlertEventPerCarMax > 0 {
145         for _, carID := range l2.CarIDs {
146             count := cfg.AlertEventPerCarMin +
147                 r.Intn(cfg.AlertEventPerCarMax-cfg.AlertEventPerCarMin+1)
148             for j := 0; j < count; j++ {
149                 lat := 59.899901 + r.Float64()*0.2
150                 lon := 30.265473 + r.Float64()*0.6
151                 inserted++
152                 _, err := tx.Exec(ctx, query,
153                     carID,
154                     l2.EmployeeIDs[r.Intn(len(l2.EmployeeIDs))],
155                     l1.AlertEventTypeIDs[r.Intn(len(l1.AlertEventTypeIDs))],
156                     lat, lon,
157                     fmt.Sprintf("Событие %d", inserted),
158                     l1.AlertProcStatusIDs[r.Intn(len(l1.AlertProcStatusIDs))],
159                     )
160                 if err != nil {
161                     return 0, fmt.Errorf("insert alert_event[%d]: %w",
162                         inserted-1, err)
163                 }
164             }
165         }
166         return inserted, nil
167     }
168
169     total := cfg.AlertEvents
170     if total <= 0 {
171         return 0, nil
172     }
173
174     base := total / len(l2.CarIDs)
175     rem := total % len(l2.CarIDs)
176     perm := r.Perm(len(l2.CarIDs))
177     for i, carIdx := range perm {
178         carID := l2.CarIDs[carIdx]
179         count := base
180         if i < rem {
181             count++
182         }
183         for j := 0; j < count; j++ {
184             lat := 59.899901 + r.Float64()*0.2

```

```

183     lon := 30.265473 + r.Float64()*0.6
184     inserted++
185     _, err := tx.Exec(ctx, query,
186         carID,
187         l2.EmployeeIDs[r.Intn(len(l2.EmployeeIDs))],
188         l1.AlertEventTypeIDs[r.Intn(len(l1.AlertEventTypeIDs))],
189         lat, lon,
190         fmt.Sprintf("Событие %d", inserted),
191         l1.AlertProcStatusIDs[r.Intn(len(l1.AlertProcStatusIDs))],
192     )
193     if err != nil {
194         return 0, fmt.Errorf("insert alert_event[%d]: %w",
195             inserted-1, err)
196     }
197 }
198 return inserted, nil
199 }
200
201 func seedGeoRequests(ctx context.Context, tx pgx.Tx, r
202     *rand.Rand, cfg Config, l1 *level1Refs, l2 *level2Refs) (int,
203     error) {
204     total := 0
205     for _, employeeID := range l2.EmployeeIDs {
206         requestCount := cfg.GeoRequestsMin +
207             r.Intn(cfg.GeoRequestsMax-cfg.GeoRequestsMin+1)
208         for i := 0; i < requestCount; i++ {
209             query := "INSERT INTO geo_request(employee_id, car_id,
210                 geo_request_type_id, request_goal) VALUES ($1, $2, $3, $4)"
211             _, err := tx.Exec(ctx, query,
212                 employeeID,
213                 l2.CarIDs[r.Intn(len(l2.CarIDs))],
214                 l1.GeoReqTypeIDIDs[r.Intn(len(l1.GeoReqTypeIDIDs))],
215                 fmt.Sprintf("Запрос сотрудника %d, запись %d", employeeID,
216                     i+1),
217             )
218             if err != nil {
219                 return 0, fmt.Errorf("insert geo_request(employee=%d,
220                     idx=%d): %w", employeeID, i, err)
221             }
222             total++
223         }
224     }
225     return total, nil
226 }

```

Listing 24: Заполнение зависимых сущностей (internal/seeder/level3.go)

```

1 package seeder
2
3 import (
4     "context"
5     "fmt"
6     "math/rand"
7
8     "github.com/jackc/pgx/v5/pgxpool"
9 )
10
11 func seedLevel4(ctx context.Context, pool *pgxpool.Pool, r
    *rand.Rand, cfg Config, tracks []trackRef, dataSourceIDs []int)
    (int, error) {
12     type row struct {
13         trackID      int
14         carID         int
15         lat, lon      float64
16         speed         float64
17         dataSourceID int
18     }
19
20     batch := make([]row, 0, cfg.BatchSize)
21     total := 0
22
23     flush := func() error {
24         if len(batch) == 0 {
25             return nil
26         }
27         tx, err := pool.Begin(ctx)
28         if err != nil {
29             return fmt.Errorf("level4 begin: %w", err)
30         }
31         defer tx.Rollback(ctx)
32
33         for _, b := range batch {
34             query := "INSERT INTO track_point(track_id, car_id,
                latitude, longitude, speed_kmh, data_source_id) VALUES ($1, $2,
                $3, $4, $5, $6)"
35             if _, err := tx.Exec(ctx, query, b.trackID, b.carID, b.lat,
                b.lon, b.speed, b.dataSourceID); err != nil {
36                 return fmt.Errorf("insert track_point: %w", err)
37             }
38         }
39         if err := tx.Commit(ctx); err != nil {
40             return fmt.Errorf("level4 commit: %w", err)
41         }
42         total += len(batch)
43         batch = batch[:0]
44         return nil
45     }
46
47     for _, tr := range tracks {

```



```

48     lat := 59.790891 + r.Float64()*0.2
49     lon := 30.324453 + r.Float64()*0.6
50     pointCount := cfg.TrackPointsMin +
r.Intn(cfg.TrackPointsMax-cfg.TrackPointsMin+1)
51     for p := 0; p < pointCount; p++ {
52         lat += (r.Float64() - 0.5) * 0.002
53         lon += (r.Float64() - 0.5) * 0.003
54         if lat < 59.8 {
55             lat = 59.8
56         }
57         if lat > 60.0 {
58             lat = 60.0
59         }
60         if lon < 30.0 {
61             lon = 30.0
62         }
63         if lon > 30.7 {
64             lon = 30.7
65         }
66
67         batch = append(batch, row{
68             trackID:      tr.ID,
69             carID:         tr.CarID,
70             lat:           lat,
71             lon:           lon,
72             speed:         float64(r.Intn(121)),
73             dataSourceID: dataSourceIDs[r.Intn(len(dataSourceIDs))],
74         })
75
76         if len(batch) >= cfg.BatchSize {
77             if err := flush(); err != nil {
78                 return 0, err
79             }
80         }
81     }
82 }
83 if err := flush(); err != nil {
84     return 0, err
85 }
86 return total, nil
87 }

```

Listing 25: Заполнение точек треков (internal/seeder/level4.go)